

Smart Football: Fair Team Allocation and Session Management Platform

Deborah Owolabi

Student ID: 100685951

Supervised by Youbing Zhao

6CM995 Individual Project

Discipline of Computing and Mathematics

School of Computing and Engineering

University of Derby

Submitted May 2026, in partial fulfilment of the conditions for the award of the degree of BSc Computer Science

Abstract

This project designed and evaluated Smart Football, a Django-based web platform for supporting fair and efficient community football organisation. The project addressed the problem that informal football sessions often rely on group chats, manual lists and organiser memory, which can make attendance tracking, session management and team selection inconsistent. Smart Football was developed as a database-driven prototype that allows players to register, manage football profiles, view and join sessions, access team allocations and review match information. Organisers can create sessions, manage attendance, generate teams, view dashboard summaries and review match results.

The system was built using Django, SQLite, HTML, CSS, JavaScript and Chart.js. A lightweight team allocation method was implemented to assign players into balanced teams using stored player information such as ratings, skill level, fitness and experience. The prototype was evaluated using synthetic data, functional test cases, permission testing, fairness comparison, usability review and performance checks.

The evaluation showed that the main workflows operated successfully, including authentication, session management, attendance tracking, team allocation, dashboards and match results. The fairness evaluation showed that the generated team allocations improved team balance in seven out of eight tested sessions compared with a simple join-order baseline. The project concludes that a lightweight web-based system can support more organised and transparent community football management, although further real-user testing, deployment and algorithm improvements would be required before production use.

Table of Contents

1 Table of Contents

Abstract.....	2
Table of Contents.....	3
List of Figures	6
List of Tables	7
List of Abbreviations.....	8
1 Introduction	9
1.1 Project Rationale	9
1.2 Problem Statement.....	9
1.3 Project Aim	10
1.4 Objectives.....	10
1.5 Research Questions	10
1.6 Scope and Limitations	10
2 Literature Review	12
2.1 Introduction.....	12
2.2 Digital Technology in Community Sport	12
2.3 Team Formation and Balancing Methods	13
2.4 Fairness, Ratings, and Trust in Allocation Systems	14
2.5 Dashboards, Visualisation, and Web System Considerations	15
2.6 Web Application Frameworks and System Considerations	16
2.7 Research Gap and Chapter Summary	16
3 Research Methodology	17
3.1 Introduction.....	17
3.2 Research Strategy	18
3.3 Tools and Technologies.....	19
3.4 Data Generation Methods.....	19
3.5 Data Analysis and Evaluation Methods.....	20
3.6 Ethics	20
3.7 Risks and Limitations.....	21
4 System Design and Implementation	23
4.1 Introduction.....	23
4.2 Requirements Overview	23

4.3	System Architecture.....	24
4.4	Database Design	25
4.5	User Interface and Navigation Design	27
4.6	Authentication and Role-Based Access	29
4.7	Match Scheduling and Capacity Control	30
4.8	Attendance Management.....	31
4.9	Team Allocation Algorithm	31
4.10	Dashboard and Data Visualisation Implementation.....	34
4.11	Match Results and Player Statistics	35
4.12	Implementation Challenges and Solutions	35
5	Testing and Evaluation	37
5.1	Introduction.....	37
5.2	Test Environment and Test Data	37
5.3	Functional Testing	38
5.4	Permission and Security Testing	40
5.5	Fairness Evaluation	41
5.6	Usability Evaluation	43
5.7	Performance Evaluation	44
5.8	Results Summary	45
5.9	Chapter Summary	46
6	Discussion.....	47
6.1	Introduction.....	47
6.2	Achievement of Aim and Objectives	47
6.3	Interpretation of Functional Testing Results	48
6.4	Interpretation of Fairness Results	49
6.5	Interpretation of Usability and Dashboard Findings.....	50
6.6	Comparison with Literature	51
6.7	Limitations and Threats to Validity	53
6.8	Objective Achievement Table	54
7	Conclusions and Recommendations	56
7.1	Conclusion.....	56
7.2	Summary of Contributions	56
7.3	Recommendations and Future Work	57
7.4	Final Reflection.....	57

8	References.....	59
9	Appendices	63
9.1	Appendix A: User Manual / Demo Guide	63
9.2	Appendix B: Screenshots and Wireframes.....	67

List of Figures

Figure 4.3.1: System architecture diagram for the Smart Football web application.....	25
Figure 4.4.1: Entity Relationship Diagram showing the main Smart Football database models and relationships.....	27
Figure 4.5.1: Player Dashboard Screenshot.....	28
Figure 4.5.2: Organiser Dashboard Screenshot	29
Figure 4.9.1: Flowchart showing the weighted greedy team allocation process used to generate balanced teams in Smart Football.....	34
Figure 5.2.1: Django system check output showing no detected issues	38
Figure 5.4.1: Permission Testing that only Organisers can create matches	41
Figure 5.5.1: Fairness Evaluation.....	42
Figure 9.6.1: Player dashboard Wireframe.....	67
Figure 9.6.2: Login/Signup wireframe.....	68
Figure 9.6.3: Login/Signup page	68
Figure 9.6.4: Team Allocation page.....	69
Figure 9.6.5: All sessions page showing available matches, filters, session status	69
Figure 9.6.6: Match details page showing session information, capacity, participants, attendance summary	70
Figure 9.6.7: Attendance management page showing player attendance statuses and organiser attendance controls.....	70
Figure 9.6.8: Match results page showing final score, attendance count, MVP, fairness score and match statistics	71
Figure 9.6.9: Edit profile page showing football profile fields such as position, skill level, fitness level and experience.	71

List of Tables

Table 1.1: Research questions for the Smart Football project	10
Table 3.1: Evaluation methods used to assess the Smart Football prototype	21
Table 4.1: Functional requirements	24
Table 5.1: Functional test cases and results	39
Table 5.2: Comparison of generated team allocations against a join-order baseline	42
Table 5.3: Usability review of core player and organiser tasks	44
Table 5.4: Performance checks for core prototype functions	45
Table 6.1: Achievement of project objectives	54

List of Abbreviations

Abbreviation	Meaning
AI	Artificial Intelligence
CSS	Cascading Style Sheets
DB	Database
ERD	Entity Relationship Diagram
HTML	HyperText Markup Language
UI	User Interface
UX	User Experience

1 Introduction

1.1 Project Rationale

Community football is an important part of local sport participation because it gives players an accessible way to stay active, socialise, and take part in organised matches without needing to belong to a formal club. However, the organisation of informal or amateur football sessions often remains dependent on manual processes such as group chats, spreadsheets, verbal confirmation, or organiser memory. Although these methods may be simple for small groups, they can become inefficient as the number of players, sessions, and organiser responsibilities increases.

Common problems in casual football organisation include uncertainty around player attendance, limited visibility of available sessions, overfilled matches, and difficulty producing balanced teams. Organisers often need to manually track who has joined a match, who actually attended, and how players should be divided into fair teams. This can create inconsistent decision-making, especially when player skill levels, preferred positions, and previous attendance are not recorded in a structured way.

Team imbalance is a particular issue because an unfair match can reduce enjoyment for both stronger and weaker players. If teams are formed informally, organisers may unintentionally favour familiar players or repeatedly place stronger players together. This can make matches less competitive, discourage participation, and reduce trust in the organisation process. A digital system that stores relevant player and session data can help reduce these problems by making match organisation more transparent and consistent.

This project addresses these issues through the design and development of Smart Football, a web-based platform for managing community football sessions. The system supports user registration, player profiles, match scheduling, attendance tracking, organiser dashboards, player dashboards, match results, and automated team allocation. The central purpose of the system is to show how a data-driven approach can support fairer and more efficient match organisation for small-sided community football.

1.2 Problem Statement

The main problem addressed by this project is that community football organisers often lack a structured digital tool for managing sessions and creating balanced teams. Existing manual methods can work for very small groups, but they become unreliable when organisers need to manage multiple players, attendance records, ratings, and team allocation decisions.

The lack of structured data creates three key issues. First, attendance is difficult to manage accurately because players may join or drop out informally. This affects planning, capacity control, and team preparation. Second, team selection can be subjective because organisers may rely on memory or personal judgement rather than consistent player information. Third, players have limited visibility of their own participation, upcoming sessions, team allocation, and match outcomes.

The project therefore focuses on creating a prototype that improves the organisation process by storing key information in a database and using it to support match management decisions. The system is not intended to replace full commercial sports management platforms. Instead, it

provides a focused academic prototype that demonstrates how session management, player data, attendance tracking, and fairness-based team allocation can be combined into one coherent web application.

1.3 Project Aim

The aim of this project is to design and evaluate a data-driven web platform that supports fair and efficient community football organisation through automated team formation, player self-assessment, attendance tracking, and scheduling tools.

1.4 Objectives

To achieve this aim, the project uses the following objectives:

1. To develop a secure login and registration system that allows users to access the platform as players or organisers.
2. To implement match scheduling functionality that allows organisers to create sessions and allows players to view, join, and leave available matches.
3. To implement attendance management so organisers can record whether players attended a session.
4. To develop a fairness-based team allocation feature that uses player profile information and ratings to create balanced teams.
5. To create dashboard and statistics pages that visualise useful information such as session attendance, participation, team allocation, and match results.
6. To evaluate the prototype in terms of functionality, usability, fairness, and performance using synthetic data and structured test cases.

1.5 Research Questions

Table 1.1: Research questions for the Smart Football project

Research Question	What it investigates
RQ1	How can a web-based system improve community football organisation compared with informal manual methods?
RQ2	How effectively can a lightweight data-driven team allocation method create balanced teams for casual football sessions?
RQ3	What dashboard information is useful for players and organisers when managing community football participation?

1.6 Scope and Limitations

The scope of this project is limited to the design, development, and evaluation of a prototype web application for managing small-sided community football sessions. The system focuses on supporting two main user roles: players and organisers. Players can register, log in, edit their profile, view available sessions, join or leave matches, view team allocations, and access match-related information. Organisers can create sessions, manage attendance, generate balanced teams, view dashboard information, and review match results.

The project includes core functionality related to match scheduling, attendance tracking, player profile management, team allocation, dashboards, and match results. These features were chosen because they directly support the project aim of improving fairness and efficiency in community football organisation. The system is designed around small-sided games, particularly 5-a-side style sessions, where player numbers are limited and team balance has a clear effect on the quality of the match.

The project deliberately excludes some features that would be required in a full commercial sports management system. Payment processing is outside the project scope, even though real community football sessions may involve small participation fees. The system also does not include live chat, push notifications, mobile app deployment, GPS-based venue features, or integration with external booking platforms. These exclusions keep the project focused on the core academic problem of fair match organisation.

Another limitation is that the system uses synthetic data for testing and demonstration rather than real player data. This was chosen to avoid collecting unnecessary personal information and to reduce ethical and privacy risks. However, synthetic data may not fully reflect the unpredictability of real football groups, such as last-minute cancellations, inaccurate self-assessment, or social preferences between players. The prototype is also evaluated in a local development environment rather than through full public deployment, so its performance and usability findings are limited to controlled testing conditions.

The project should be understood as a functional academic prototype. It demonstrates how a data-driven system can support fairer team allocation and more structured session management, but further development and real-world testing would be needed before it could be used as a production system.

2 Literature Review

2.1 Introduction

2.2 Digital Technology in Community Sport

Digital technology is increasingly relevant to community sport because participation depends not only on interest in physical activity, but also on access, organisation, communication, and ease of use. In small-scale grassroots settings, session organisation may rely on informal methods such as group chats, spreadsheets, verbal confirmation, or organiser memory. While these methods may work for very small groups, they can become unreliable when organisers need to manage attendance, capacity, player information, and team balance across multiple sessions.

The FA's *Grassroots Strategy 2024–2028* highlights the importance of grassroots football in England, describing it as a way to unite communities, tackle inequalities, and improve health (The Football Association, 2024). The strategy also projects further growth in the grassroots game, with around 15,000 more teams expected over four years (The Football Association, 2024). This growth creates a need for stronger organisational support, especially for volunteers and local organisers. Although The FA recognises digital improvement as part of grassroots football development, its strategy mainly focuses on national priorities such as facilities, participation pathways, volunteers, behaviour, and equal opportunities. It does not directly address smaller operational problems such as session visibility, attendance tracking, or balanced team formation.

Sport England's latest Active Lives data also shows why community sport remains a current issue. The report states that 64.6% of adults in England, around 30.9 million people, met the recommended activity level between November 2024 and November 2025 (Sport England, 2026). However, participation remains unequal, with lower activity levels among women, disabled people, older adults, some ethnic groups, and people from lower socio-economic backgrounds (Sport England, 2026). This suggests that community sport systems should reduce unnecessary barriers by making participation easier to access and manage.

Academic research shows that digital sport tools are not automatically successful simply because they exist. Qian and Kim (2025) argue that sport participants' intention to use digital technology is shaped by both reasons for adoption and reasons against adoption. Their study found that barriers to adoption can have a stronger effect than positive reasons for adoption, while perceived enjoyment and ease of use remain important factors (Qian and Kim, 2025). This is related to Smart Football because the platform must be simple, useful, and not create extra work for users.

Harris and Crone (2020) also found that engagement with a technology-enabled community physical activity intervention was influenced by accessibility, social influence, awareness, and practical barriers. This supports the idea that community sport technologies should be designed around real user behaviour rather than assuming that people will use a system just because it exists. Therefore, Smart Football addresses a focused gap by providing a lightweight web platform for session visibility, attendance tracking, player data, and fair team allocation in community football.

2.3 Team Formation and Balancing Methods

Team formation is a key technical issue in Smart Football because the system must divide available players into fair teams using limited but meaningful data. In this project, fairness is mainly based on balancing skill level, preferred position and attendance availability. This links the project to wider computational team formation problems, where people with different attributes must be allocated into groups while satisfying constraints. In football, this matters because a team is not simply a group of players, but a combination of roles, abilities and balance. Beal, Norman and Ramchurn (2019) explain that artificial intelligence research in team sports considers decisions such as team selection, tactics and player performance, showing that team composition is a recognised computational challenge in sport.

A useful way to understand team balancing is through partitioning. In a partitioning problem, a set of items is divided into groups so that a chosen measure is as equal as possible across those groups. In Smart Football, the items are players and the main weight is their skill rating. The aim is to split players into two teams with similar total skill scores while also considering basic position distribution. Recalde et al. (2018) apply balanced k-way partitioning to a sports realignment problem, showing how sports allocation can be modelled using weight constraints and exact optimisation methods. Although their work focuses on league realignment rather than individual sessions, it supports the idea that fair sports grouping can be treated as a formal optimisation problem rather than organiser guesswork.

Team balancing is also related to matchmaking. Alman and McKay (2017) argue that fair team games should consider both fairness and uniformity. Fairness means that teams should have a roughly equal chance of winning, while uniformity means avoiding large differences between the strongest and weakest players in a match. At grassroots level, a game can feel unfair even when both teams have the same number of players, if one side has most of the strongest players. Smart Football uses skill ratings as a practical proxy for team strength, helping create allocations that are more balanced than random or fully manual selection.

However, research also shows that team formation becomes computationally difficult when more constraints are added. Vombatkere, Gionis and Terzi (2025) study team formation as a problem of balancing skill coverage and expert workload and show that their balanced coverage formulation is NP-hard. Barnabò et al. (2019) similarly show that fair team formation with skill and fairness constraints is computationally difficult. This matters because a complex optimiser would be harder to implement and understand.

Different algorithmic approaches therefore involve trade-offs. Exact methods, such as integer programming, can search for an optimal solution, but they need a precise model and may be excessive for a lightweight web application. Metaheuristic methods, such as genetic algorithms, can explore many possible team combinations, but they require tuning and may be less transparent. Sanchez-Anguix et al. (2023) argue that computational team formation often depends on evaluation heuristics because real team performance is difficult to measure before the activity takes place. This supports the use of a simple approximation rather than claiming that the system can perfectly predict match outcomes.

For this reason, Smart Football uses a greedy team balancing approach. Players can be sorted by skill rating from highest to lowest, then assigned one by one to the team with the lower current total skill score. This does not guarantee a perfect global optimum, because early

choices can affect later allocation. However, it is fast, understandable and suitable for small-sided football sessions. It also allows organisers to review and manually adjust the final teams where needed. Therefore, the algorithm is best described as decision support rather than machine learning: it uses stored player data to support fairer allocation while keeping the process transparent.

2.4 Fairness, Ratings, and Trust in Allocation Systems

Fairness in allocation systems is not only about producing numerically balanced outcomes. In Smart Football, an allocation may appear fair if both teams have a similar combined skill score, but users may still question the result if they do not understand how players were assigned. This matters for Smart Football because the system uses player profile data, attendance information and ratings to support team formation. If these inputs are incomplete, subjective or poorly explained, the allocation may be perceived as unfair even if the final team scores appear balanced.

The Centre for Data Ethics and Innovation (2020) argues that algorithmic decision-making can improve fairness, but it can also reinforce existing bias if data, design choices or decision processes are not managed carefully. The report explains that fairness is difficult to define because different fairness goals can conflict with one another. It also distinguishes between procedural fairness, which concerns how a decision is made, and outcome fairness, which concerns the result of the decision. This distinction is relevant to Smart Football because a fair team allocation should not only produce balanced teams but should also use an understandable and consistent process. Therefore, fairness in this project must include transparent criteria, such as skill level, position preference, attendance record and ratings, rather than relying on hidden or unexplained decisions.

Ratings add another fairness challenge because they rely on human judgement. Power and Tanner (2023) found that peer assessment can be relatively reliable when compared with expert assessment, but peer scores may still be inflated. They also found that self-assessment had lower reliability than peer assessment. Smart Football may use both self-reported player information and post-match ratings. These inputs can improve team allocation by giving the system more information about players, but they should not be treated as completely objective. Players may overestimate their own ability, rate friends more generously, or give lower ratings based on personal frustration rather than performance. For this reason, ratings should support allocation decisions rather than fully control them.

Fairness also depends on how users perceive the decision-making process. Starke et al. (2022) argue that perceptions of algorithmic fairness are shaped by multiple factors, including the decision outcome, the process used, the features included, the context of the decision and whether explanations are provided. This suggests that Smart Football should make its allocation logic visible to organisers and players where appropriate. For example, showing team skill totals, player positions and a fairness score can help users understand why the system generated a particular allocation.

Trust is also linked to transparency. Aysolmaz, Müller and Meacham (2023) found that transparency concerns influence perceived fairness, accountability and privacy, which then affect trust, perceived usefulness and intention to adopt algorithmic decision-making systems.

For Smart Football, this means users are more likely to trust automated team allocation if the system is understandable and if organisers can review or adjust the generated teams. Fairness in Smart Football should therefore be defined as a combination of balanced outcomes, reliable data, transparent reasoning and organiser oversight. This makes the allocation process more trustworthy than informal team selection based only on memory, friendship or subjective judgement.

2.5 Dashboards, Visualisation, and Web System Considerations

Dashboards are relevant to Smart Football because organisers need to understand session information quickly without manually checking separate records. Few (2006) defines a dashboard as a visual display of the most important information needed to achieve one or more objectives, arranged so it can be monitored. This supports the design of Smart Football's organiser and player dashboards, where key information such as upcoming sessions, attendance rates, player numbers, ratings and match results is summarised in one place. Rather than presenting data only as tables, dashboard cards and charts help users identify important patterns more quickly.

Visualisation is also important in football decision-making. Atterling (2025) found that interactive visualisation tools can support football decision-making by improving transparency, scenario exploration and strategic reflection. Although Atterling's work focuses on professional football squad planning, the same principle applies at a smaller scale in community football. Smart Football uses dashboard summaries and team allocation information to help organisers review the state of a session before making decisions. However, visualisation should support judgement rather than replace it, especially because casual football involves human factors such as attendance reliability, player confidence and positional preference.

Existing sports management platforms show that there is already demand for digital tools in team organisation. TeamSnap provides features such as roster management, scheduling, communication, availability tracking and payment support (TeamSnap, n.d.). Spond also supports team and club management through event planning, participant management, messaging, reminders, file storage and payment collection (Spond, n.d.). Pitchero focuses on club management, offering tools for club websites, membership management, payments, fixtures, results, news and team communication (Pitchero, n.d.). These platforms demonstrate that sports organisers often need digital support for administration, communication and coordination.

However, these commercial systems are mainly designed for established teams, clubs or larger sports organisations. Their public feature descriptions focus strongly on administration, communication, payments, membership and fixture management. They do not clearly present skill-based team allocation for informal community football sessions as a central feature. This creates a practical gap for Smart Football. The project is not intended to compete with full club management platforms. Instead, it focuses on a narrower problem: helping small football

organisers create sessions, track attendance, manage basic player data and generate balanced teams.

The literature and existing platforms suggest that dashboards and digital tools can improve sports organisation when they present useful information clearly. However, existing systems are often broader than the needs of casual football groups. Smart Football addresses this gap by combining dashboard summaries, session management and fair team allocation in a lightweight web-based prototype.

2.6 Web Application Frameworks and System Considerations

The choice of web framework is important because Smart Football requires user accounts, database models, role-based access, form handling, dashboards and session management. Django was selected because it provides a structured framework for building database-driven web applications. Kumar and Nandal (2024) argue that Django supports rapid web application development through features such as its Model-Template-View architecture, Object-Relational Mapping, reusable components and administrative functionality. This is relevant to Smart Football because the system includes several connected models, including users, player profiles, match sessions, participation records, ratings, team assignments and match results.

Django is also suitable because it provides more built-in structure than a lightweight framework such as Flask. Idris et al. (2020) compare Django and Flask as web technologies, showing that both can be used for web development but differ in structure and approach. Flask is lightweight and flexible, while Django provides more built-in functionality. For Smart Football, Django's structure is useful because the prototype needs authentication, database interaction, templates and permissions without requiring every component to be created from scratch. This makes Django more appropriate for a time-limited dissertation project with multiple connected features.

Authentication and access control are especially important because Smart Football has different user roles. Players should be able to view sessions, join matches and edit profiles, while organisers require additional access to create sessions, mark attendance and generate teams. Django's authentication system supports user accounts, password handling, login, permissions and groups, making it suitable for separating player and organiser functionality (Django, 2026a). Django also provides built-in protections against common web security risks, including cross-site scripting, cross-site request forgery and SQL injection, although developers must still validate input and configure deployment securely (Django, 2026b).

Privacy is also relevant because Smart Football stores user and participation data. The ICO's data minimisation principle states that personal data should be adequate, relevant and limited to what is necessary for the stated purpose (Information Commissioner's Office, n.d.). Therefore, Smart Football should only collect information needed for authentication, session management, attendance tracking and team allocation. Django provides a suitable technical foundation for building a secure, structured and database-driven community football management prototype.

2.7 Research Gap and Chapter Summary

This chapter reviewed literature and existing systems related to community sport technology, team formation, fairness, dashboards, and web application development. The literature shows that digital technology can support community sport by improving access, organisation, and engagement. National sport strategies recognise the importance of grassroots football and

digital development, while participation research highlights the need to reduce barriers to physical activity (The Football Association, 2024; Sport England, 2026). However, these sources mainly focus on broad participation, national development, and access to sport rather than the practical problems faced by small football organisers.

The review also showed that team balancing can be understood as a computational problem. Existing research on team sports, team formation, partitioning, and matchmaking demonstrates that fair allocation can be supported using data-driven methods (Beal, Norman and Ramchurn, 2019; Alman and McKay, 2017; Barnabò et al., 2019; Sanchez-Anguix et al., 2023). However, many of these approaches are designed for more complex or theoretical settings. They may be too advanced for a lightweight community football prototype where organisers need a fast, understandable and reviewable method rather than a hidden optimisation process.

Fairness was also identified as a key issue. Algorithmic fairness research shows that fairness is not only about outcomes, but also about process, transparency, bias, accountability and user trust (Centre for Data Ethics and Innovation, 2020; Starke et al., 2022; Aysolmaz, Müller and Meacham, 2023). This is relevant because a football team allocation can be numerically balanced but still be questioned by users if the reasoning is unclear. Research on peer and self-assessment also shows that ratings can provide useful input, but they should not be treated as fully objective because human judgement can be inconsistent or inflated (Power and Tanner, 2023).

The review of dashboards and existing sports management platforms showed that digital tools already support sport administration through scheduling, communication, membership, payments and availability tracking (TeamSnap, n.d.; Spond, n.d.; Pitchero, n.d.). Dashboard design literature also supports the use of clear visual summaries to help users monitor important information quickly (Few, 2006). However, existing commercial platforms appear to focus more on formal clubs, structured teams and administration than on lightweight skill-based team allocation for informal football sessions.

Therefore, the research gap addressed by Smart Football is the lack of a focused, web platform that combines session scheduling, attendance tracking, player profile data, dashboard summaries and transparent team allocation for small-scale community football. Smart Football responds to this gap by developing a Django-based prototype that supports both players and organisers, uses stored player data to generate balanced teams, and presents useful information through dashboards and match management pages. This provides a practical contribution by showing how fairness-focused team allocation can be integrated into a community football session management system.

3 Research Methodology

3.1 Introduction

This chapter explains the methodology used to design, develop and evaluate the Smart Football prototype. The project followed a design-and-build approach because the main outcome was a working web application rather than a purely hypothetical investigation. The methodology

focuses on how the system was planned, implemented, tested and evaluated against the project objectives.

Smart Football was developed as a Django-based web platform for community football session management and fair team allocation. The system includes user registration, profile management, match scheduling, attendance tracking, dashboard visualisations, ratings, match results and automated team allocation. Since the project combines software development with evaluation, the methodology needed to cover both the construction of the artefact and the collection of evidence to assess whether it met its intended aims.

The chapter outlines the research strategy and development approach then explains the main tools and technologies used including Django, SQLite, HTML, CSS, JavaScript, Chart.js and Git. The chapter also explains how synthetic data was created for testing, how the system was evaluated and what ethical considerations were applied. Finally, it discusses risks, limitations and how these were managed during the project.

3.2 Research Strategy

The development of Smart Football followed an Agile-inspired iterative methodology. This was appropriate because the project involved designing and building a working software artefact with several connected features, including authentication, player profiles, match scheduling, attendance tracking, team allocation, dashboards and match results. Agile development is suitable for this type of project because it supports incremental delivery, regular review and adaptation when requirements change. The Agile Manifesto principles emphasise early and continuous delivery of valuable software, responding to change, and delivering working software frequently (Beck et al., 2001). These ideas matched the needs of the project because the system had to be developed in stages rather than completed all at once.

The project was divided into development iterations. The first iteration focused on the minimum viable product, including Django setup, user authentication, basic match sessions and joining sessions. This created the foundation for later features. The second iteration added capacity checks, leaving sessions, attendance toggling and organiser-specific functionality. The third iteration introduced player profiles, ratings and team assignment models. Later iterations focused on the team allocation algorithm, dashboard visualisations, match results and frontend improvements. This staged approach made the system easier to test because each new feature could be checked before adding further complexity.

The methodology was Agile inspired rather than a full Scrum process. Scrum uses an iterative and incremental approach to improve predictability and control risk, with sprints and inspection/adaptation points (Schwaber and Sutherland, 2020). However, Smart Football was an individual dissertation project rather than a team-based commercial project, so formal Scrum roles such as Scrum Master and Product Owner were not used. Instead, the project used short development cycles, feature prioritisation, testing after each major addition, and refinement based on implementation problems and project objectives.

This approach was useful because many features depended on earlier work. For example, attendance tracking required users and participation records, team allocation required player profiles and match participants, dashboards required stored session, attendance, rating and result data. Building incrementally reduced the risk of trying to integrate everything at the end.

The Agile inspired process therefore supported the project by allowing the prototype to evolve from a basic scheduling system into a more complete community football management platform.

3.3 Tools and Technologies

The main backend framework used for Smart Football was Django. Django was chosen because it provides a structured framework for building database-driven web applications. Django documentation explains that its model layer provides an abstraction for structuring and manipulating application data (Django, 2026a). This was useful for Smart Football because the system required several connected data models, including user profiles, match sessions, participation records, ratings, team assignments and match results.

Django was also used because it provides built-in support for authentication and authorisation. The Django authentication documentation states that Django provides user objects, password handling, permissions, groups and tools for limiting access to logged-in users (Django, 2026b). This supported the project objective of creating secure login and role-based access for players and organisers.

SQLite was used as the development database. SQLite documentation describes SQLite as a self-contained, serverless and zero-configuration SQL database engine (SQLite, 2025). This made it appropriate for a local academic prototype because it avoided the complexity of configuring a separate database server. Python's sqlite3 documentation also notes that SQLite can be useful for prototyping applications before moving to a larger database system if needed (Python, 2026a). This fits the scope of Smart Football because the project was evaluated locally rather than deployed as a production system.

The frontend was developed using HTML, CSS and JavaScript through Django templates. MDN Web Docs describes HTML, CSS and JavaScript as core technologies for structuring content, styling pages and adding interactive behaviour on the web (Mozilla, 2025). Chart.js was used for dashboard charts because it provides common chart types and customisation options for web-based visualisation (Chart.js, 2025). This supported the dashboard objective by allowing attendance, participation and match-related statistics to be presented visually.

A Python virtual environment was used to isolate project dependencies. The Python documentation explains that venv creates lightweight virtual environments with their own installed packages, separate from the base Python installation (Python, 2026b). This helped keep the project environment controlled and reproducible.

3.4 Data Generation Methods

Synthetic data was used for development, testing and demonstration. This included player accounts, organiser accounts, player profiles, match sessions, participation records, attendance values, ratings, team assignments, match results and player statistics. Synthetic data was chosen because the project did not require real user data to demonstrate the prototype's core functionality.

The dataset was designed to reflect realistic community football scenarios. Player profiles included varied skill levels, fitness levels, experience levels and preferred positions. Match sessions included different dates, capacities and locations. Participation records were used to show players joining matches, while attendance values were used to test organiser attendance

tracking. Ratings and match results were included to support dashboard statistics and post-match review features.

Using synthetic data had two main benefits. First, it allowed controlled testing because the same data could be reused across different test scenarios. Second, it reduced ethical and privacy risks because no real player information was collected. However, synthetic data is also a limitation because it may not fully represent real football behaviour, such as last-minute cancellations, inaccurate self-assessment, friendship preferences or disagreements about ratings.

3.5 Data Analysis and Evaluation Methods

The prototype was evaluated using four main methods: functional testing, permission testing, fairness evaluation and usability/performance review.

Functional testing was used to check whether the main system features worked as expected. Test cases were created for key user journeys, including registration, login, profile editing, viewing sessions, joining and leaving sessions, viewing match details, marking attendance, generating teams, viewing dashboards and viewing match results. Each test compared the expected result with the actual system behaviour.

Permission testing was used to check whether organiser-only features were protected. This was important because normal players should not be able to create matches, mark attendance or generate teams for sessions they do not organise. Django's authentication and permission features supported these checks by allowing views to restrict access based on the logged-in user and their role (Django, 2026b).

Fairness evaluation was used to assess the team allocation feature. The main measure was the difference between the total scores of Team A and Team B. A smaller difference indicated a more balanced allocation. The evaluation also considered whether the allocation was understandable to users by showing team totals, player scores and a fairness score. This was important because the literature review showed that fairness is not only about numerical balance, but also transparency and trust.

Usability review was used to check whether the interface supported common player and organiser tasks. This included reviewing the consistency of navigation, clarity of buttons, dashboard layout, session pages and profile forms. Performance was reviewed through local testing, focusing mainly on whether pages loaded successfully and whether team generation was fast enough for small-sided football sessions.

3.6 Ethics

The main ethical issue in this project was the handling of user and participation data. Although Smart Football was developed as a prototype, the system still represents data such as names, profiles, attendance, ratings and match participation. For this reason, the project used synthetic data rather than collecting real personal data from football players.

The project also followed the principle of data minimisation. The ICO states that personal data should be adequate, relevant and limited to what is necessary for the stated purpose (Information Commissioner's Office, n.d.). In Smart Football, this means the system should only store data needed for authentication, session management, attendance tracking and team

allocation. Unnecessary personal details, such as addresses, phone numbers or sensitive personal information, were not required for the prototype.

Security was also considered during development. Django's security documentation states that developers should not trust user-controlled data and should validate input, while Django provides protections against common risks such as cross-site scripting, cross-site request forgery and SQL injection (Django, 2026c). Smart Football used Django forms, CSRF tokens and ORM-based database queries to support safer handling of user input.

The evaluation was carried out in a local development environment using synthetic data. This reduced risk because no real users were exposed to the system and no real personal information was processed. However, if the system were developed beyond prototype stage, further ethical review, privacy notices, user consent processes and stronger deployment security would be required.

3.7 Risks and Limitations

Several risks affected the methodology. The first risk was time. Smart Football includes multiple connected features, and there was a risk that adding too many features would reduce the quality of the final prototype. This was managed by focusing on the core project objectives: authentication, scheduling, attendance, team allocation, dashboards and evaluation.

The second risk was integration complexity. Features such as dashboards and team allocation depend on data from several models. If the relationships between users, profiles, sessions, participation records and team assignments were incorrect, the system would not work reliably. This was managed through incremental development and repeated testing after each major feature was added.

The third risk was algorithmic limitation. The team allocation method uses a lightweight scoring approach, so it cannot fully represent real football ability. It also depends on profile data and ratings, which may be subjective or incomplete. This was managed by presenting the algorithm as decision support rather than a perfect prediction system.

A further limitation is that testing used synthetic data and local development conditions. This means the evaluation cannot fully prove how the system would behave with real users, real clubs, multiple simultaneous users or deployment on a public server. The prototype works in a controlled academic setting, but further testing would be needed before production use.

Table 3.1: Evaluation methods used to assess the Smart Football prototype

Evaluation area	Method	Evidence to include	Success measure
Functionality	Manual/functional test cases	Test-case table, screenshots	Core journeys pass
Fairness	Compare generated teams using balance metrics	Skill totals, rating differences, fairness score	Generated teams are more balanced than baseline/random split
Usability	Heuristic review or small user tasks	Task completion notes, feedback	Users can complete main tasks with minimal confusion

Evaluation area	Method	Evidence to include	Success measure
Performance	Timing checks	Team generation time, page load notes	Runs fast enough for typical 5-a-side/small sessions

4 System Design and Implementation

4.1 Introduction

This chapter presents the design and implementation of the Smart Football prototype. The purpose of this chapter is to explain how the system was structured, how the main features were implemented, and how the technical design supports the project aim and objectives. While the previous chapters explain the problem and methodology, this chapter focuses on the actual artefact produced for the project.

Smart Football was designed as a web-based platform for organising community football sessions and creating fairer team allocations. The system supports two main user roles: players and organisers. Players can register, log in, edit their football profile, view available sessions, join or leave matches, view team allocations, and access match results. Organisers have additional permissions that allow them to create match sessions, manage attendance, generate teams, and view organiser-focused dashboard information.

The implementation uses Django as the main backend framework. Django was selected because it provides built-in support for user authentication, database models, URL routing, views, templates, and form handling. These features made it suitable for building a structured prototype with role-based access and database-driven functionality. SQLite was used as the development database because it is lightweight and appropriate for a local prototype. The frontend was implemented using Django templates, HTML, CSS, and JavaScript, with Chart.js used for dashboard visualisations.

The chapter is organised around the main design areas of the system. It begins with an overview of functional and non-functional requirements. It then explains the system architecture, database design, user interface structure, authentication and role-based access, match scheduling, attendance management, team allocation, dashboards, match results, and implementation challenges. Together, these sections show how the prototype was built to address the project objectives.

4.2 Requirements Overview

The requirements for Smart Football were based on the project aim, the identified problems in community football organisation, and the features needed to demonstrate a working prototype. The system was designed to support the core activities of organising small-sided football sessions: registering users, managing player profiles, creating matches, joining sessions, recording attendance, generating teams, and displaying useful statistics.

The functional requirements describe what the system must allow users to do. These requirements were divided between player features and organiser features. Player features focus on participation, profile management, and visibility of match information. Organiser features focus on session control, attendance management, team allocation, and dashboards. This separation was important because the system needed to reflect the different responsibilities of each user type.

The non-functional requirements focus on qualities such as usability, security, performance, and maintainability. Since the system is a dissertation prototype rather than a production platform, the non-functional requirements were kept realistic and measurable. The interface needed to be

simple enough for players and organisers to understand without training. Access control needed to prevent normal players from using organiser-only functions. The team generation process needed to run quickly for typical small-sided football session sizes. The code also needed to be modular enough to explain and test during the dissertation.

Table 4.1:Functional requirements

Requirement ID	Requirement
FR1	Users can register, log in, and log out
FR2	Players can edit football profile
FR3	Organisers can create sessions
FR4	Players can join and leave sessions
FR5	The system prevents over-capacity joins
FR6	Organisers can manage attendance
FR7	The system can generate balanced teams
FR8	Users can view dashboard statistics
FR9	Users can view completed match results

4.3 System Architecture

Smart Football uses a traditional server-side web application architecture. The user interacts with the system through a web browser. Requests from the browser are sent to Django URL routes, which direct each request to the appropriate view function. The view function processes the request, retrieves or updates data through Django models, and then returns a rendered HTML template to the browser.

The system is organised into Django apps. The accounts app manages user-related functionality such as registration, login, logout, and player profile editing. The scheduling app manages football-specific functionality such as match sessions, participation, attendance, ratings, team assignments, dashboards, and results. This separation makes the system easier to maintain because authentication and football session management are handled in separate areas of the codebase.

The backend communicates with an SQLite database through Django's Object Relational Mapping system. Instead of writing raw SQL for each database action, the system uses Django models to represent database tables. For example, the MatchSession model represents a football session, while the Participation model links users to the sessions they have joined. This model-based approach keeps the database logic more readable and reduces the chance of inconsistent queries.

The frontend is built using Django templates. These templates combine HTML with Django template syntax so that database values can be displayed dynamically. For example, the session list page displays available matches from the database, and the match details page displays participants, attendance status, and generated teams for a selected session. CSS is used to create a consistent visual design across the platform, while Chart.js is used for charts and dashboard visualisations.

The team allocation feature sits between the session management data and the user interface. It reads player data from the database, calculates a score for each participant, assigns players into Team A and Team B, stores the result in the TeamAssignment model, and then displays the generated teams on the relevant page. This allows team allocation to be treated as part of the wider system rather than as a separate offline calculation.

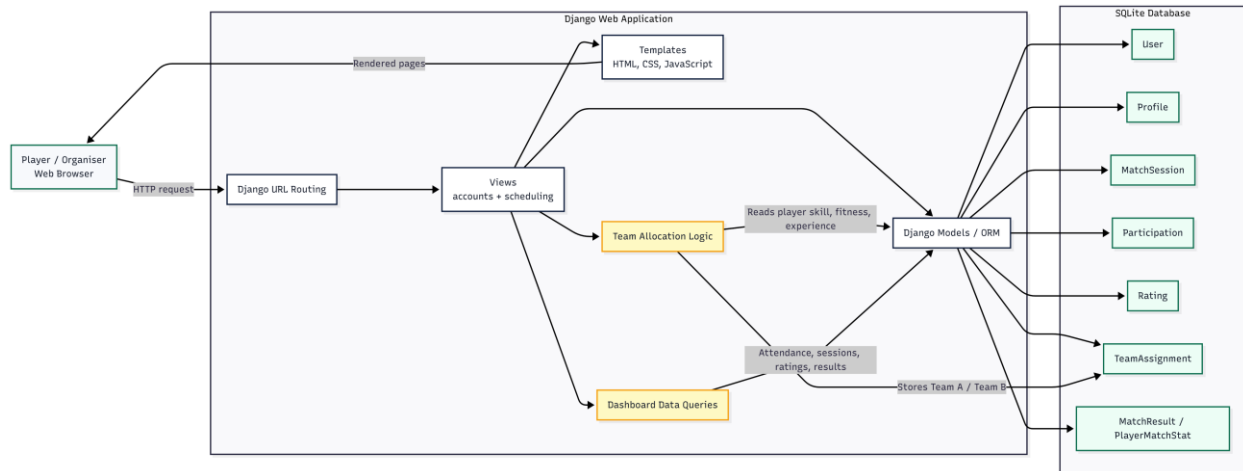


Figure 4.3.1: System architecture diagram for the Smart Football web application

4.4 Database Design

The database design supports the main football organisation features of the platform. Django's built-in User model is used for authentication and stores basic account information such as username, password, email, first name, and last name. This is extended through a Profile model, which stores football-specific information about each user. The profile includes fields such as role, skill level, fitness level, preferred position, and experience. These fields are important because they are later used to support team allocation and player filtering.

The MatchSession model represents a football match or session created by an organiser. Each match session stores information such as the title, start date and time, location, capacity, and the user who created it. The created_by field links each session to an organiser account. This relationship is important because it allows the system to check whether the current user is allowed to manage attendance or generate teams for that session.

The Participation model links users to match sessions. This model is used when a player joins a session. It stores the session, the user, and an attendance status. A uniqueness constraint prevents the same user from joining the same session more than once. This is important for data accuracy because the dashboard statistics, attendance counts, and capacity checks all rely on correct participation records.

The Rating model stores feedback between users after sessions. It records the session, the user giving the rating, the user receiving the rating, a score, an optional comment, and the creation date. This makes it possible to include peer feedback as part of the wider football profile and evaluation process. Ratings can also be used to support future improvements to the team allocation logic.

The TeamAssignment model stores the result of the team generation process. Each assignment links a session, a user, and a team label such as Team A or Team B. This means generated teams are not temporary values shown only once on screen. Instead, they are stored in the database and can be viewed again later from the match details or team allocation pages.

The project also includes match result and player match statistic functionality. Match results store completed match outcomes, while player match statistics store individual performance values such as goals, assists, and ratings. These models support the match results page and help demonstrate how the system could be extended beyond scheduling into participation history and performance tracking.

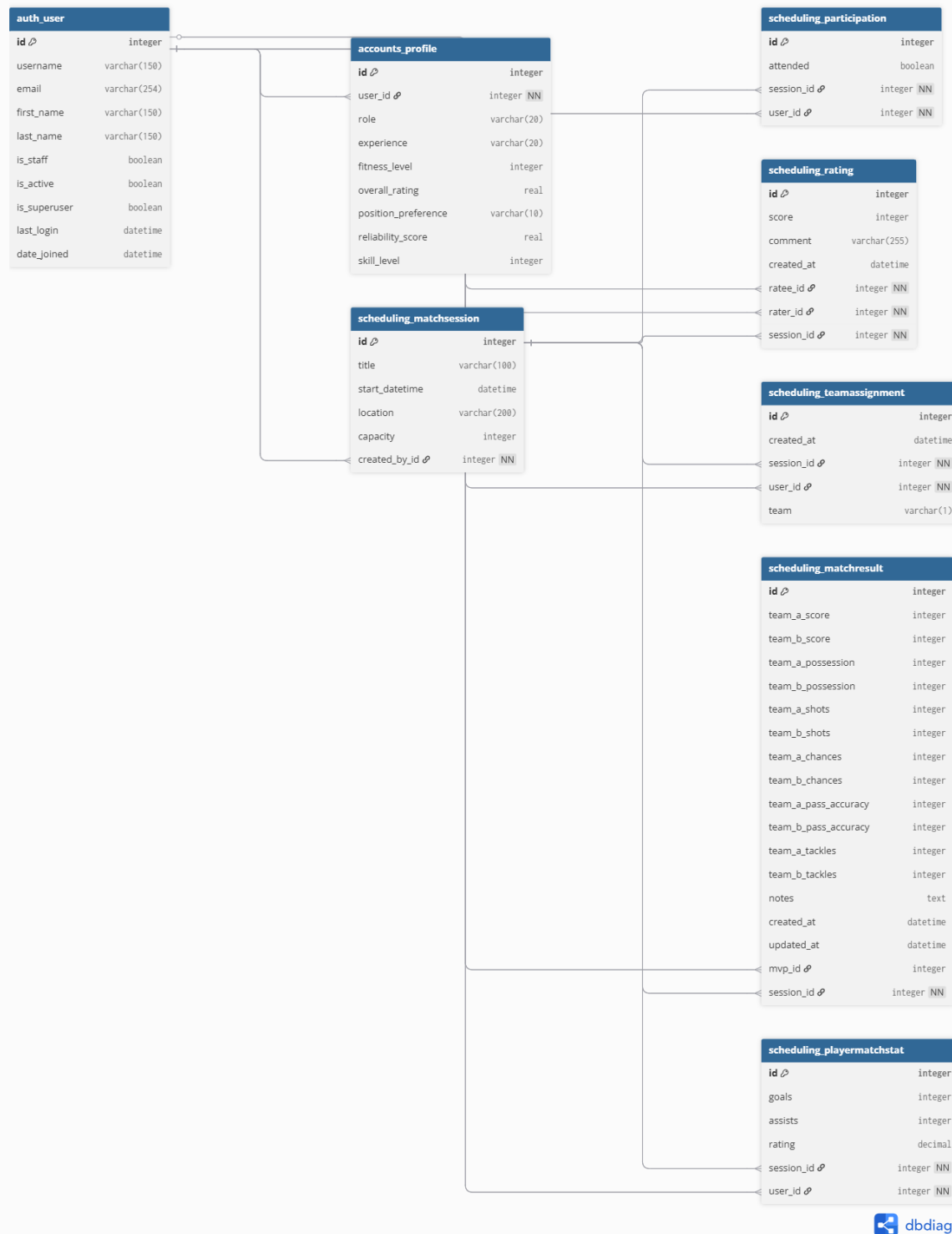


Figure 4.4.1: Entity Relationship Diagram showing the main Smart Football database models and relationships

4.5 User Interface and Navigation Design

The user interface was designed to make the main football organisation tasks easy to access. A dashboard-style layout was used because the system needs to display practical information

such as upcoming sessions, attendance statistics, player details, generated teams, and match results. The interface uses a consistent navigation structure with links to the dashboard, all sessions, profile page, and organiser tools where appropriate.

The visual design follows a clean sports management style. The colour scheme uses pitch green as the main brand colour, deep navy for navigation and structure, lime green for important action buttons, white for cards and forms, and grey for borders and muted text. Status colours are used consistently, such as green for success or attended states, amber for warnings, red for full or error states, and blue for informational dashboard metrics. This helps users understand page states quickly without needing long explanations.

The player dashboard acts as the starting point for normal users. It summarises the player's joined matches, attendance rate, rating information, upcoming sessions, and profile details. The 'all sessions page' allows users to browse available matches, filter sessions, view capacity, and join sessions. The match details page then provides deeper information about a selected session, including date, location, capacity, participants, attendance information, generated teams, and actions such as joining, leaving, rating, or viewing results.

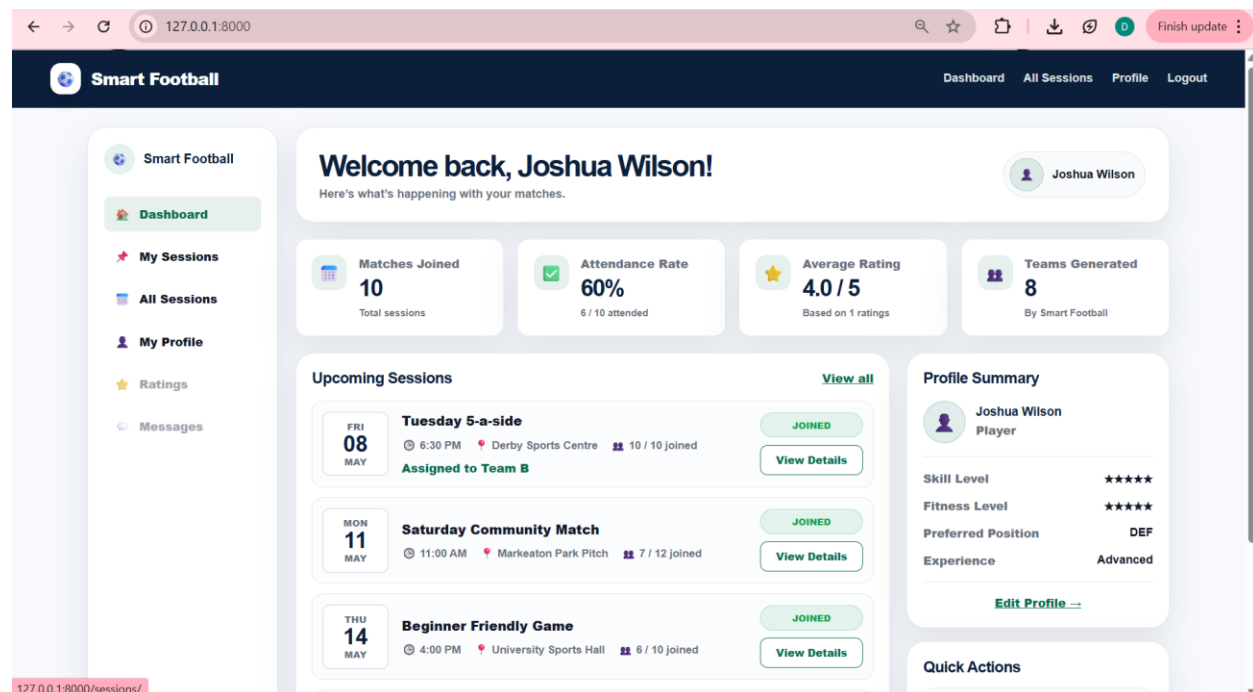


Figure 4.5.1: Player Dashboard Screenshot

The organiser-facing interface provides additional controls. Organisers can create sessions, manage attendance, generate teams, and view dashboard statistics related to their sessions. Organiser-only actions are not placed everywhere in the interface because this would make the navigation cluttered. Instead, features such as attendance management and match results are accessed from relevant match detail pages. This keeps the sidebar simpler while still making the tools available in context.

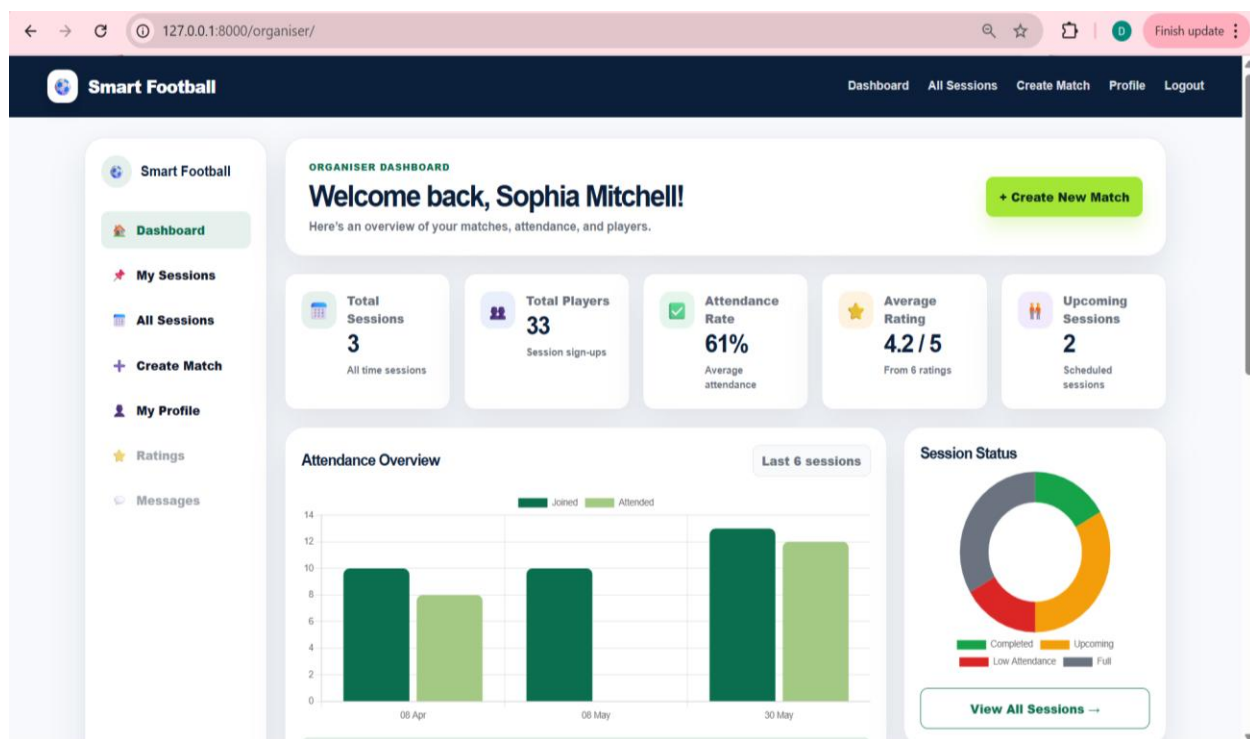


Figure 4.5.2: Organiser Dashboard Screenshot

Wireframes were used to plan the main pages before implementation. These included the login/sign-up page, player dashboard, match details page, edit profile page, create match page, organiser dashboard, all sessions page, team allocation page, feedback page, attendance management page, and match results page. The implemented interface follows these planned flows while adapting details where needed to match the actual Django models and database fields.

4.6 Authentication and Role-Based Access

Smart Football uses Django's built-in authentication system to manage registration, login, logout, and session access. This was selected because Django provides secure password handling, user session management, and authentication checks without requiring these features to be implemented manually. Users can create an account, log in using their username and password, and log out securely through the navigation bar.

The authentication flow begins at the combined login and sign-up page. New users can create an account and are then directed to complete their player profile. Existing users can log in and access the dashboard. Once authenticated, users can access pages such as the player dashboard, all sessions page, match details page, and edit profile page. Unauthenticated users are restricted from actions such as joining sessions, leaving sessions, editing profiles, or accessing dashboard features.

A profile model is used to extend Django's default user model with football-specific information. This includes fields such as role, skill level, fitness level, preferred position, experience, reliability score and rating. Separating the profile from the authentication model keeps account data separate from football participation data. This also makes it easier to use player-specific information in features such as team allocation and dashboards.

Role-based access is important because players and organisers have different responsibilities. Normal players can view sessions, join or leave matches, edit their profile, and view generated teams. Organisers have additional access to create sessions, manage attendance, generate teams, and view organiser-focused dashboard information. The system checks whether the current user is the organiser of a session before allowing organiser-only actions such as marking attendance or generating teams.

Access restrictions are implemented in the Django views. For example, actions that change data are protected using login checks, and organiser-only functions return a forbidden response if the logged-in user does not have the correct permission. This helps prevent normal players from changing attendance records or generating teams for matches they did not create. These checks support the security requirement of the system by ensuring that users can only perform actions appropriate to their role.

4.7 Match Scheduling and Capacity Control

The match scheduling feature allows organisers to create football sessions that players can later view and join. Each match session stores the session title, date and time, location, capacity, and organiser. This information is stored in the MatchSession model and displayed across the session list, match details page, player dashboard, and organiser dashboard.

The create match page allows an organiser to enter the basic details of a session. Once submitted, the session is stored in the database and becomes visible to users on the 'all sessions page'. The 'all sessions page' provides a structured overview of available and completed matches. It displays key information such as match title, date, time, location, capacity, current number of participants, and status. This makes it easier for players to decide which sessions they want to join.

Players can join a session from either the session list or match details page. When a user joins, the system creates a Participation record linking that user to the selected MatchSession. Players can also leave a session, which removes their participation record. This supports flexibility because casual football players may need to change their availability before a match.

Capacity control was implemented to prevent sessions from becoming overfilled. Before a new participation record is created, Smart Football checks the number of players already joined to the session and compares this with the session capacity. If the maximum capacity has already been reached, the join action is blocked, and no additional participation record is created. This matters in a small-sided football setting because overfilled sessions can make planning harder, reduce match quality and affect the reliability of generated team allocations. The session detail page also separates upcoming and completed session behaviour. Upcoming sessions allow players to join or leave, while completed sessions are more relevant for results, ratings and statistics. This distinction helps the system support both pre-match organisation and post-match review.

4.8 Attendance Management

Attendance management is included to help organisers record which players attended a football session. This is necessary because joining a session does not always mean that a player attended the match. In informal football groups, players may join but later fail to attend, which affects team planning, reliability, and future decision-making. By recording attendance, Smart Football can provide a more accurate view of participation.

Attendance is stored in the Participation model using an attended field. When a player joins a session, a participation record is created with attendance initially set as false. The organiser can later update this attendance status from the match details or attendance management interface. This makes attendance part of the same data structure as session participation, rather than storing it separately.

Only the organiser of a session can mark attendance. This restriction is important because attendance records should be controlled by the person responsible for managing the match. If normal players could mark themselves or other users as attended, the data could become unreliable. The system therefore checks whether the logged-in user is the creator of the session before allowing attendance changes.

Attendance information is used in several areas of the system. It is displayed on the match details page so that the organiser can see which players have been marked as attended. It can also be used in dashboards to calculate attendance rates and participation summaries. This supports the project objective of creating useful statistics for organisers and players.

The attendance feature also contributes to the fairness aim of the project. A player who regularly joins but does not attend could affect the reliability of team planning. Although the current prototype uses attendance mainly for tracking and display, the data structure allows future versions to include attendance reliability as part of team generation or organiser decision-making.

4.9 Team Allocation Algorithm

The team allocation feature is the main fairness-focused part of Smart Football. Its purpose is to reduce the subjectivity involved in manually selecting teams by using stored player information to generate two balanced teams. The system uses data from player profiles and participation records to calculate a simple score for each player before assigning them to Team A or Team B.

The algorithm begins by retrieving all players who have joined a selected match session. For each participant, the system reads football-related profile data such as skill level, fitness level, and experience. These values are used to calculate a player score. In the prototype, skill level is weighted strongly because it is the clearest indicator of playing ability. Fitness level and experience are also included because they may affect player performance during a match.

After each player score is calculated, the players are sorted from highest to lowest score. The algorithm then assigns players one at a time to the team with the lower current total score. This means stronger players are distributed across both teams instead of being placed together. The aim is to reduce the difference between the total score of Team A and Team B.

The generated teams are saved in the TeamAssignment model. This is important because the result of the algorithm is stored in the database rather than only displayed temporarily. Once saved, the teams can be shown again on the match details page, team allocation page, and relevant dashboard areas. If the organiser regenerates the teams, the previous assignments are removed and replaced with the new team allocation.

The algorithm is intentionally lightweight because the project focuses on small-sided community football rather than large-scale professional optimisation. It does not attempt to solve every possible constraint, such as friendship groups, left-footed players, defensive balance, or repeated team combinations. However, it provides a transparent and understandable method that is suitable for a dissertation prototype. It also creates a foundation for future improvements, such as adding position constraints, reliability scores, historical ratings, or optimisation methods.

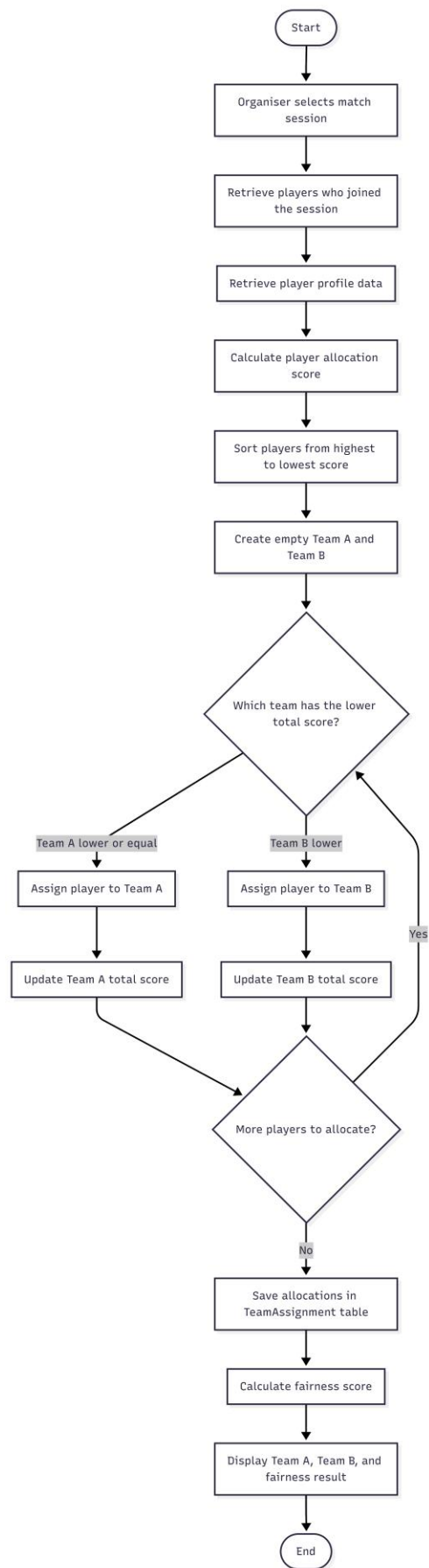


Figure 4.9.1: Flowchart showing the weighted greedy team allocation process used to generate balanced teams in Smart Football

4.10 Dashboard and Data Visualisation Implementation

The dashboard features were implemented to provide users with a clear summary of relevant football session data. Smart Football includes both player-focused and organiser-focused dashboards. These dashboards support the project objective of visualising useful match and attendance statistics.

The player dashboard is designed as the main landing page for logged-in users. It displays information such as matches joined, attendance rate, average rating, teams generated, upcoming sessions, and profile summary. This gives players a quick overview of their own participation and helps them access common actions such as viewing sessions, joining matches, or editing their profile.

The organiser dashboard focuses on session management and overview statistics. It allows organisers to monitor created sessions, attendance information, participation levels, and other match-related data. This is useful because organisers need a quick way to understand how their sessions are performing without manually checking each match one by one. The organiser dashboard therefore supports better decision-making and reduces the amount of manual tracking needed.

The dashboard data is generated using database queries in Django views. For example, participation records are used to calculate how many matches a player has joined and how many they attended. Rating records are used to calculate average rating values. Match session and participation records are used to display upcoming sessions and capacity information. This means the dashboards are not static screens; they reflect live values from the database.

Chart.js is used for visualising selected dashboard metrics. Charts help make statistics easier to interpret than raw numbers alone. For example, attendance trends, skill balance, session participation, and result summaries can be shown visually. The project uses a consistent chart colour palette so that visualisations match the rest of the interface and remain readable.

The dashboards also support evaluation because they provide visible evidence that database values are being stored, calculated, and displayed correctly. In Chapter 5, screenshots and test cases can be used to show that the dashboard figures match the underlying database records. This links the implementation directly to the project's evaluation criteria.

The implemented dashboard interfaces are shown earlier in Figure 4.5.1 and Figure 4.5.2. Figure 4.5.1 shows the player dashboard, which presents player-focused information such as joined matches, attendance rate, average rating, upcoming sessions and profile summary. Figure 4.5.2 shows the organiser dashboard, which presents organiser-focused information such as session activity, player numbers, attendance summaries, dashboard statistics and alerts. These figures provide visual evidence that the dashboard data is generated from stored Smart Football database records and displayed in a format that supports quick monitoring and decision-making.

4.11 Match Results and Player Statistics

The match results feature was added to support post-match review and to make the system more complete as a football session management platform. While the earlier parts of the system focus on organising sessions before they happen, the results feature allows completed matches to be recorded and reviewed afterwards. This supports the project aim by connecting match scheduling, team allocation, attendance, ratings, and performance information within one platform.

The match result functionality stores team-level information for completed sessions. This includes the scores for Team A and Team B, as well as additional match statistics such as possession, shots, chances created, pass accuracy, tackles, and match notes. These values help describe the outcome of the match in more detail than the final score alone. The system also includes an MVP field, allowing one player to be highlighted for their performance.

Player match statistics are stored separately from the match result. This allows individual users to be linked to their personal performance in a completed match. The system can store values such as goals, assists, and player rating. This structure makes it possible to show both team-level outcomes and individual player contributions. It also creates a foundation for future improvements, such as using player statistics to update ratings or influence future team allocation.

The match results page is designed to appear after completed games rather than as a general sidebar feature. This design decision keeps the navigation simpler and places results in the context where users would expect them: after viewing a match that has already happened. Both organisers and players can view match results, which supports transparency and allows players to review previous outcomes.

This feature also supports evaluation. Completed match records provide useful evidence that the system can store and display more than just session bookings. By including results and player statistics, Smart Football demonstrates how a community football system could develop into a wider participation history and performance tracking platform. However, the current version is still limited because the results are manually entered rather than automatically captured during a match.

4.12 Implementation Challenges and Solutions

Several implementation challenges occurred during the development of Smart Football. These challenges were mainly related to integrating multiple features into one coherent Django application. The project was not only a collection of separate pages; authentication, profiles, sessions, attendance, ratings, team allocation, dashboards, and match results all needed to work together using the same database structure. Several issues appeared at the points where separate features interacted, particularly where authentication, session data, participation records and team assignments depended on one another.

One challenge was maintaining consistent navigation across the application. As new pages were added, such as the player dashboard, organiser dashboard, match details, attendance management, and match results pages, the sidebar and navbar became inconsistent between templates. This was solved by simplifying the navigation and making sure the most important

pages appeared consistently across the system. Some organiser-only functions were also moved into the relevant match details page rather than being placed directly in the sidebar. This made the interface cleaner and reduced unnecessary clutter.

Another challenge was role-based access. Normal players should not be able to perform organiser-only actions such as managing attendance or generating teams for sessions they did not create. This was solved by adding checks in the relevant Django views. The system verifies whether the logged-in user is the creator of the session before allowing organiser actions. If the user does not have permission, the system blocks the action. This helped ensure that organiser features were not only hidden visually but also protected at the backend level.

A further challenge was linking the team allocation feature to real profile and participation data. The algorithm needed to retrieve players who had joined a specific session, read their profile values, calculate player scores, assign them to teams, and save the result. This required the Participation, Profile, and TeamAssignment models to work together correctly. The solution was to keep the algorithm lightweight and transparent, using skill level, fitness level, and experience as the main inputs. This made the feature easier to test and explain within the dissertation.

Seed data was also important because the prototype needed enough realistic data to demonstrate dashboards, attendance, ratings, teams, and match results. Manually creating all users, sessions, and participation records would have been slow and inconsistent. A Django-based data seeding approach was therefore used so that demo users and football records could be created safely. This was especially important for user accounts because Django passwords need to be hashed correctly rather than inserted as plain SQL values.

Finally, maintaining UI consistency became a challenge as the application grew. Early pages were basic, while later pages used dashboard-style cards, tables, status badges, and charts. This was addressed by creating a shared colour scheme and reusable CSS classes for buttons, cards, forms, tables, badges, and dashboard layouts. The final interface therefore appears more consistent across login, dashboard, session list, match details, profile, attendance, team allocation, and result pages.

5 Testing and Evaluation

5.1 Introduction

This chapter presents the testing and evaluation of the Smart Football prototype. The purpose of this chapter is to assess whether the implemented system works correctly and whether it meets the project objectives identified in Chapter 1. The evaluation focuses on functionality, permission and security behaviour, team allocation fairness, usability, and performance.

The testing process was based on the main user journeys within the system. These included logging in, registering, editing a profile, viewing sessions, joining and leaving a session, viewing match details, marking attendance, generating teams, viewing dashboards, and reviewing match results. These workflows were selected because they represent the core features required for a community football session management platform.

The chapter uses evidence from the running Django application, including screenshots, test case results, fairness values, and system checks. Functional testing was used to confirm that the main features behaved as expected. Permission testing was used to check that organiser-only features were protected from normal players. Fairness evaluation was used to assess whether generated teams were reasonably balanced. Usability and performance were also reviewed to determine whether the prototype was practical for small-sided football session management.

The chapter presents the results factually. The wider meaning of these results, including limitations and links back to the project aim and research questions, is discussed in Chapter 6.

5.2 Test Environment and Test Data

Testing was carried out in a local development environment using the Django development server. The project was run from Visual Studio Code using a Python virtual environment. The system was accessed through a web browser at <http://127.0.0.1:8000/>. SQLite was used as the development database. This environment was suitable for prototype testing because the system was designed as a local academic artefact rather than a deployed production application.

Before functional testing, the Django system check command was run using `python manage.py check`. The result showed that the system check identified no issues. This provided initial confirmation that Django did not detect configuration, model, URL, or application-level problems at the time of testing.

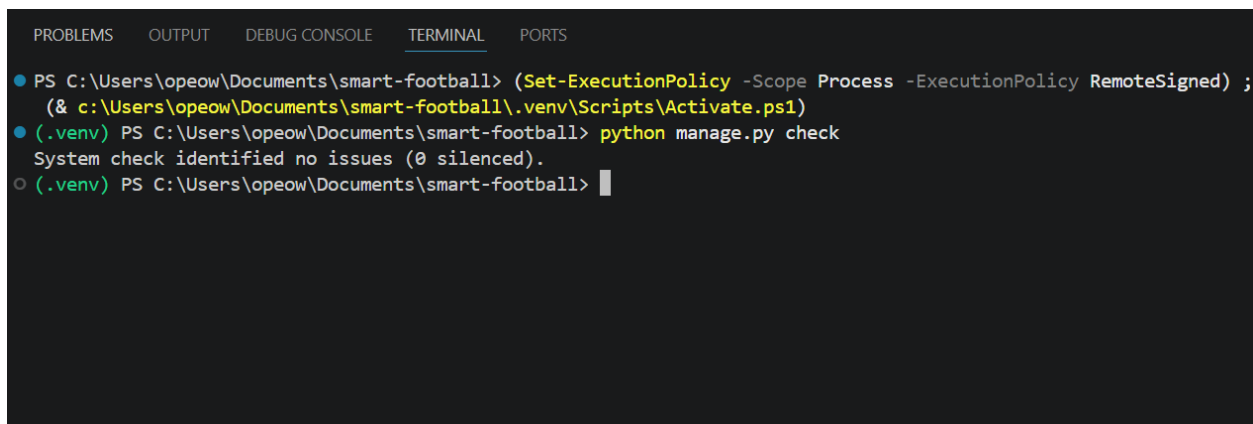
A screenshot of a terminal window with a dark background. At the top, there are tabs labeled 'PROBLEMS', 'OUTPUT', 'DEBUG CONSOLE', 'TERMINAL', and 'PORTS'. The 'TERMINAL' tab is active. The terminal shows a PowerShell prompt 'PS C:\Users\opeow\Documents\smart-football>' followed by the command '(Set-ExecutionPolicy -Scope Process -ExecutionPolicy RemoteSigned) ; (& c:\Users\opeow\Documents\smart-football\.venv\Scripts\Activate.ps1)'. The prompt changes to '(.venv) PS C:\Users\opeow\Documents\smart-football>' and the command 'python manage.py check' is entered. The output is 'System check identified no issues (0 silenced)'. The prompt returns to '(.venv) PS C:\Users\opeow\Documents\smart-football>'.

Figure 5.2.1: Django system check output showing no detected issues

Synthetic test data was used for the evaluation. This included organiser accounts, player accounts, player profiles, match sessions, participation records, attendance data, team assignments, ratings, match results, and player statistics. Synthetic data was chosen to avoid collecting real personal data from users and to allow controlled testing of different scenarios. The data included varied player skill levels, positions, fitness levels, experience values, attendance states, and match outcomes.

The use of synthetic data allowed the system to be tested with realistic football scenarios without creating privacy risks. However, it also limits the evaluation because synthetic users may not behave in the same unpredictable ways as real community football players. This limitation is discussed further in Chapter 6.

5.3 Functional Testing

Functional testing was used to check whether the main features of Smart Football behaved as expected. The tests were based on the core workflows required by the project objectives. Each test involved carrying out a task in the browser and comparing the actual result with the expected result.

The login and sign-up functionality were tested first because all other user features depend on authentication. The system allowed users to log in using valid account details and redirected authenticated users to the appropriate dashboard. The profile editing feature was tested by updating football-related player information such as skill level, fitness level, preferred position, and experience. These profile values were then used elsewhere in the system, including the dashboard and team allocation feature.

Session management was tested by viewing the 'all sessions page', opening a match details page, joining sessions, leaving sessions, and checking capacity information. The 'all sessions page' displayed available sessions, filters, match status, attendance percentage, and join actions. The match details page displayed the selected session, capacity, participant list, attendance information, and generated teams.

Attendance management was tested using organiser functionality. The attendance page displayed players assigned to a session, their team, attendance status, and an action button to

mark them as attended. This confirmed that attendance could be managed through the interface and stored as part of the session participation record.

The team allocation feature was tested by opening the team allocation page and generating teams for a session. The system displayed Team A and Team B, player names, positions, ratings, total team ratings, and a fairness breakdown. This confirmed that team assignments were generated and displayed in a way that could be reviewed by an organiser.

The match results page was also tested. It displayed the final score, attendance count, total goals, MVP, fairness score, and match statistics. This showed that the system could support post-match review as well as pre-match scheduling.

Table 5.1: Functional test cases and results

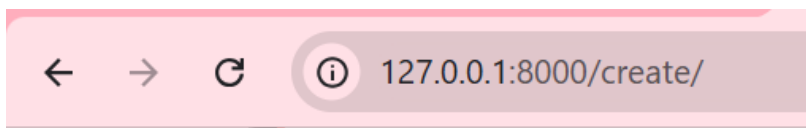
Test ID	Scenario	Steps	Expected result	Actual result	Pass/Fail
TC1	User login	Open login page, enter valid username and password, submit form	User is logged in and redirected to dashboard	User accessed dashboard successfully	Pass
TC2	User registration	Open sign-up page, enter valid details, submit form	New account is created	Sign-up page accepted user details	Pass
TC3	View player dashboard	Log in as player and open dashboard	Dashboard shows matches joined, attendance rate, rating, upcoming sessions, and profile summary	Dashboard displayed player statistics and upcoming sessions	Pass
TC4	View all sessions	Open /sessions/ page	Sessions list displays available matches and filtering options	All sessions page displayed sessions, status, attendance, and actions	Pass
TC5	View match details	Click a session from all sessions page	Match details page shows capacity, participants, attendance, and teams	Match details page displayed selected session information	Pass
TC6	Join session	Click Join on an available session	User is added to session participation list	User joined session successfully	Pass
TC7	Leave	Click Leave	User is removed	User left session	Pass

	session	Session on joined session	from session participation list	successfully	
TC8	Prevent non-organiser match creation	Access create match page as non-organiser	System blocks access	System displayed "Only organisers can create match sessions"	Pass
TC9	Mark attendance	Open attendance page as organiser and click Mark Attended	Player attendance status updates	Attendance status could be marked from organiser interface	Pass
TC10	Generate teams	Open team allocation page and generate teams	Team A and Team B are generated and displayed	Teams were generated with fairness score and team totals	Pass
TC11	View match results	Open completed match results page	Final score, attendance, MVP, and statistics are displayed	Match results page displayed score and post-match data	Pass
TC12	Django system check	Run python manage.py check	No issues are detected	System check identified no issues	Pass

5.4 Permission and Security Testing

Permission testing was carried out to confirm that restricted organiser actions could not be accessed by normal player accounts. This was important because features such as creating matches, marking attendance, and generating teams should only be available to users with organiser permissions.

One permission test involved attempting to access the create match page using an account that did not have organiser privileges. The system blocked the action and displayed the message "Only organisers can create match sessions." This confirmed that the feature was protected by backend permission logic rather than only being hidden from the navigation menu.



Only organisers can create match sessions.

Figure 5.4.1: Permission Testing that only Organisers can create matches

This result is important because hiding a link in the interface is not enough to secure a web application. A user could still manually type a URL into the browser. The permission test showed that the system checks the user's role when the restricted page is requested. This supports the security and role-based access requirements of the system.

The testing also checked that authenticated users could access player features such as the dashboard, sessions page, match details page, and profile page. Organiser accounts were able to access organiser features such as the organiser dashboard, attendance management, team allocation, and match creation. This confirms that the system separates player and organiser responsibilities appropriately.

5.5 Fairness Evaluation

The fairness evaluation focused on the team allocation feature. The purpose of this evaluation was to check whether the generated teams were reasonably balanced based on the player information available in the system. The prototype uses a lightweight balancing method rather than a complex optimisation algorithm. The algorithm considers player profile information such as skill, fitness, experience, position, and rating data, then distributes players into two teams.

The main fairness measure used in the evaluation was the difference between the total rating or score of Team A and Team B. A smaller difference suggests that the teams are more balanced. The team allocation page also displayed a fairness score to summarise how evenly matched the teams were.

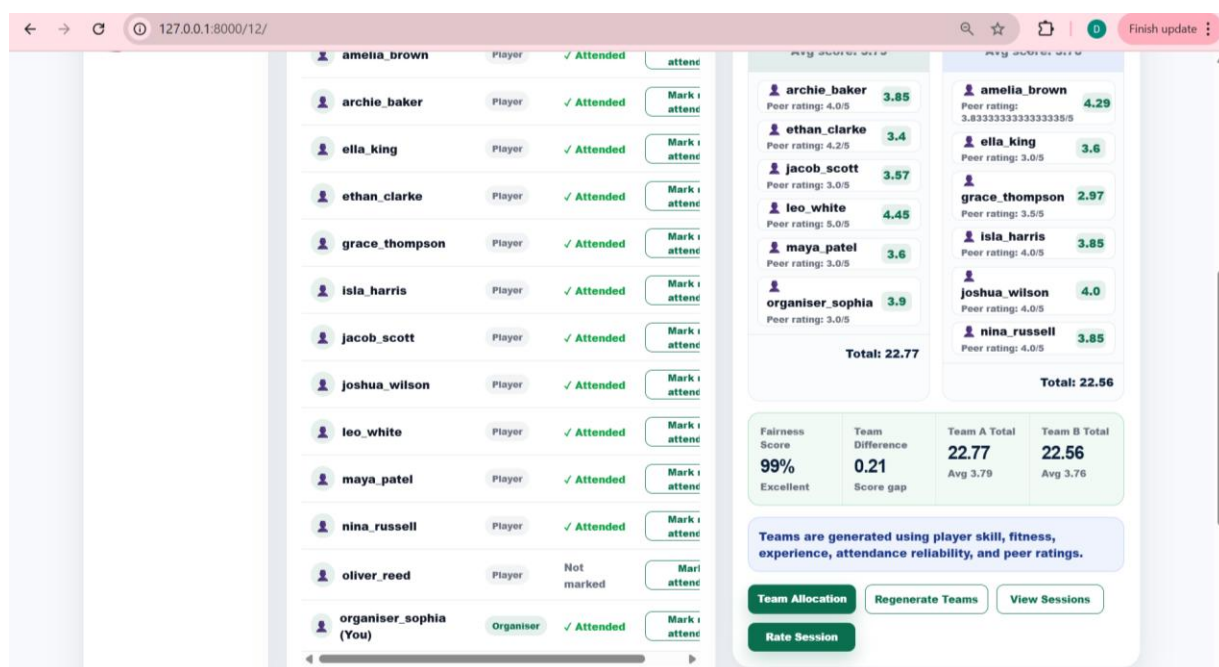


Figure 5.5.1: Fairness Evaluation

To make the fairness evaluation more reliable, the generated allocation was compared against a simple baseline allocation. The baseline used join order, where the first half of joined players were placed into Team A and the second half into Team B. This represents a basic manual method that does not actively balance player ratings. The generated allocation was then compared with this baseline using the difference between Team A and Team B total ratings. A smaller difference indicates a more balanced allocation.

The evaluation used player rating as the main comparison measure because this value was stored in the database and displayed in the team allocation feature. For each tested session, the total rating for Team A and Team B was calculated, and the absolute difference between the two totals was recorded. The same calculation was then applied to the baseline allocation so that Smart Football's generated allocation could be compared against a simple non-balancing method.

The results show that Smart Football generally reduced the difference between team totals when compared with the baseline allocation. This suggests that the team allocation feature can support fairer team formation for small-sided football sessions when suitable player rating data is available. However, the evaluation also shows that the algorithm is not perfect. The quality of the result depends on the available player data, the distribution of ratings, and whether the stored team assignments contain valid player records. Therefore, the allocation feature should be treated as decision support rather than a fully automatic replacement for organiser judgement.

Table 5.2: Comparison of generated team allocations against a join-order baseline

Session	Players	Generated Team A total	Generated Team B total	Generated difference	Baseline difference	Outcome
Monday	10	20.4	20.3	0.1	0.9	Improved

Night 5-a-side						
Community Cup Practice	10	20.8	20.9	0.1	0.5	Improved
Riverside Friendly	9	17.4	20.4	3.0	5.8	Improved
Friday Skills Match	10	19.0	18.8	0.2	0.6	Improved
Derby Social Football	8	15.7	15.8	0.1	2.9	Improved
Late Evening Kickabout	11	20.6	24.2	3.6	6.4	Improved
Tuesday 5-a-side	10	22.3	22.5	0.2	0.2	No change
Advanced Balance Trial	10	22.2	22.5	0.3	0.9	Improved

5.6 Usability Evaluation

A usability review was carried out by checking whether the main user journeys could be completed through the interface without unnecessary steps or confusion. The review focused on common tasks for players and organisers, including logging in, viewing sessions, joining a match, viewing match details, editing a profile, generating teams, marking attendance, and viewing match results.

The interface uses a consistent layout across most pages. The sidebar and top navigation provide access to key areas such as the dashboard, all sessions, profile, team allocation, attendance, ratings, and logout. This consistency helps users understand where they are in the system and reduces the need to learn a different layout for each page. Important actions such as joining a session, leaving a session, generating teams, marking attendance, and creating a match are shown as clear buttons.

The player dashboard provides a useful starting point because it summarises joined matches, attendance rate, average rating, upcoming sessions, and profile information. This supports players who need quick access to their own football activity. The organiser dashboard provides a broader management overview, including total sessions, total players, attendance rate, average rating, upcoming sessions, attendance charts, session status, and alerts. This supports organisers who need to monitor several sessions and players.

Some usability limitations were also identified. In some screenshots, the page is vertically long and requires scrolling to see all content. This is acceptable for a dashboard-style application, but future development could improve responsiveness and reduce scrolling on smaller screens. Another limitation is that some error and permission pages are plain compared with the styled interface. For example, the organiser-only permission message works correctly but does not

visually match the rest of the system. This does not affect functionality, but it could affect perceived polish.

The usability review suggests that the main workflows are understandable and that the dashboard-style interface supports both player and organiser tasks. However, further testing with real users would be needed to properly measure usability beyond developer-led review.

Table 5.3: Usability review of core player and organiser tasks

Task	User type	Expected outcome	Result	Notes
Log in	Player/organiser	User reaches dashboard	Completed	Login page provides clear entry point
View all sessions	Player/organiser	Sessions list is displayed	Completed	Filters and status labels support browsing
Join a session	Player	Participation record is created	Completed	Join button visible when session is not full
View match details	Player/organiser	Session information is displayed	Completed	Page shows participants, capacity, attendance, and teams
Generate teams	Organiser	Team A and Team B are displayed	Completed	Fairness score and team totals support review
Mark attendance	Organiser	Attendance status updates	Completed	Attendance page provides clear action buttons
View results	Player/organiser	Completed match result is displayed	Completed	Results page shows score and player statistics

5.7 Performance Evaluation

Performance evaluation was carried out to check whether the prototype was responsive enough for typical small-sided community football use. Since Smart Football is a local dissertation prototype, the evaluation focused on practical responsiveness rather than large-scale server benchmarking.

The most important performance area was the team allocation feature. This feature retrieves participants for a selected session, reads their profile data, calculates player scores, sorts players, assigns them to teams, saves the assignments, and displays the results. For small-sided football sessions, the expected number of players is usually between 10 and 16. The algorithm is lightweight because it uses simple scoring and sorting rather than expensive optimisation techniques.

During testing, the team allocation page generated teams quickly for the tested session sizes. The page loaded successfully and displayed the generated teams, total ratings, and fairness score. This indicates that the current implementation is suitable for the scale of the prototype.

The system check also confirmed that Django detected no application-level issues during testing.

A limitation of this performance evaluation is that testing was carried out locally with synthetic data. Local testing does not measure the impact of real deployment conditions such as network latency, multiple simultaneous users, server limits, or larger databases. Future work should include deployed testing and repeated timing measurements using larger datasets.

Table 5.4: Performance checks for core prototype functions

Test	Data size	Expected result	Actual result	Pass/Fail
Django system check	Full project	No system issues detected	No issues detected	Pass
Load player dashboard	Synthetic user data	Dashboard loads and displays values	Loaded successfully	Pass
Load all sessions page	12+ sessions	Sessions and filters display	Loaded successfully	Pass
Generate teams	10–12 players	Teams generated and displayed	Generated successfully	Pass
Load match results	Completed match data	Results and statistics display	Loaded successfully	Pass

5.8 Results Summary

The testing showed that the main features of Smart Football were implemented successfully. The Django system check identified no issues, which provided initial confirmation that the project configuration and application structure were valid. Functional testing showed that users could log in, view dashboards, browse sessions, view match details, join sessions, manage attendance, generate teams, and view match results.

Permission testing confirmed that restricted organiser features were protected from normal player accounts. The test where a non-organiser attempted to access the create match page showed that the backend blocked the action and displayed an appropriate permission message. This supports the role-based access requirement of the system.

The fairness evaluation showed that the team allocation feature generally improved team balance when compared with a simple join-order baseline. Across the evaluated sessions, the generated allocation reduced the difference between Team A and Team B total ratings in seven out of eight cases and produced no change in one case. This suggests that the algorithm can support fairer team formation for small-sided sessions, although the result depends on the quality of the stored player rating data and does not account for every football-specific constraint.

The usability review found that the main pages follow a consistent dashboard-style design. The player dashboard, organiser dashboard, all sessions page, match details page, team allocation page, attendance page, and results page all provide clear information and visible action buttons.

However, the review also identified areas for improvement, including styling plain permission pages and reducing scrolling on longer dashboard pages.

The results show that the prototype meets the main functional requirements and provides evidence for the project objectives. The system supports secure access, scheduling, attendance tracking, team allocation, dashboards, and match results. Further testing with real users and deployed conditions would be required before the system could be considered production ready.

5.9 Chapter Summary

This chapter evaluated the Smart Football prototype using functional testing, permission testing, fairness evaluation, usability review, and performance checking. The evaluation was based on the main objectives of the project and used evidence from the running Django application.

The functional testing showed that the main system workflows operated successfully. Users could access the authentication page, log in, view dashboards, browse sessions, view match details, and interact with the main football management features. Organisers could access additional functions such as attendance management and team allocation.

The permission testing showed that restricted organiser functions were protected from normal players. This was important because Smart Football uses different user roles, and organiser-only actions should not be available to all users. The permission blocked screenshot provided evidence that backend access control was working.

The fairness evaluation showed that the team allocation feature generally produced more balanced teams than a simple join-order baseline. The comparison of generated team totals and baseline differences provided evidence that the allocation method can support fairer team formation in the prototype. However, the evaluation also recognised that fairness is limited by the quality of player profile data, rating values and the simplified assumptions used by the algorithm.

The usability and performance review showed that the system was clear enough to support the main user journeys and responsive enough for local prototype testing. The next chapter discusses these results in more depth, linking them back to the project aim, objectives, research questions, and limitations.

6 Discussion

6.1 Introduction

This chapter discusses the results of the Smart Football prototype evaluation and explains what they mean in relation to the project aim, objectives and research questions. While Chapter 5 presented the testing and evaluation results, this chapter interprets those results and considers whether the system successfully addressed the problem identified at the start of the project.

The discussion focuses on five main areas. First, it considers how far the project aim and objectives were achieved. Second, it interprets the functional testing results and explains what they show about the reliability and completeness of the prototype. Third, it discusses the fairness evaluation and whether the team allocation feature improved team balance compared with a simple baseline method. Fourth, it considers the usability and dashboard findings, focusing on whether the interface supports player and organiser tasks effectively. Finally, the chapter compares the findings with the literature reviewed in Chapter 2 and considers the limitations and threats to validity.

The purpose of this chapter is not to claim that Smart Football is a finished commercial system. Instead, it evaluates the prototype as an academic artefact and discusses how well it demonstrates the potential of a lightweight web-based system for managing community football sessions and supporting fairer team allocation.

6.2 Achievement of Aim and Objectives

The aim of this project was to design and evaluate a data-driven web platform that supports fair and efficient community football organisation through automated team formation, player self-assessment, attendance tracking and scheduling tools. Within the scope of a local academic prototype, this aim was achieved. Smart Football provides a working web application that allows users to register, manage football profiles, view and join sessions, manage attendance, generate teams, view dashboards and review match results.

Objective 1 was to develop a secure login and registration system that allows users to access the platform as players or organisers. This was achieved through Django authentication, login, logout, registration and profile functionality. Testing showed that users could log in and access the appropriate dashboard, while role-based restrictions separated player and organiser actions.

Objective 2 was to implement match scheduling functionality that allows organisers to create sessions and allows players to view, join and leave available matches. This was achieved through the MatchSession and Participation models, session list pages, match detail pages, join actions, leave actions and capacity control. The functional tests showed that users could browse sessions, join matches and leave sessions successfully.

Objective 3 was to implement attendance management so organisers could record whether players attended a session. This was achieved through attendance status fields and organiser-only attendance controls. Testing showed that attendance could be marked from the organiser interface and stored as part of the participation record.

Objective 4 was to develop a fairness-based team allocation feature that uses player profile information and ratings to create balanced teams. This objective was mostly achieved. Smart Football generated teams using stored player data and saved the results in the TeamAssignment model. The fairness evaluation showed that the generated allocation improved team balance in most tested sessions when compared with a simple join-order baseline. However, this objective remains limited by the quality of synthetic player data and the simplified nature of the allocation algorithm.

Objective 5 was to create dashboard and statistics pages that visualise session attendance, participation, team allocation and match results. This was achieved through player and organiser dashboard pages, summary cards, tables and Chart.js visualisations. These pages displayed database-driven information and supported both player and organiser workflows.

Objective 6 was to evaluate the prototype in terms of functionality, usability, fairness and performance using synthetic data and structured test cases. This was achieved through the evaluation presented in Chapter 5. Functional testing, permission testing, fairness comparison, usability review and performance checks provided evidence that the prototype worked as intended in a controlled local environment. However, the evaluation was limited because it used synthetic data and did not include full real-world deployment or large-scale user testing.

The project achieved its main aim and objectives as a functional prototype. The strongest evidence comes from the successful implementation of core workflows and the fairness comparison against a simple baseline. The main weaknesses are the limited realism of synthetic data, the lack of real user testing and the simplified nature of the team allocation algorithm.

6.3 Interpretation of Functional Testing Results

The functional testing results suggest that Smart Football meets the main requirements expected from a working prototype. The tested workflows covered the most important actions in the application, including user login, registration, dashboard access, viewing sessions, joining and leaving matches, viewing match details, managing attendance, generating teams and viewing match results. These workflows were selected because they link directly to the project objectives and represent the main tasks that players and organisers need to complete.

The successful login and registration tests show that the authentication flow works as required. This matters for the prototype because authentication controls whether users can access the correct player or organiser workflows. The dashboard tests also show that Smart Football can display user-specific information after login, including joined sessions, attendance records, ratings, match status and dashboard summaries.

The session management tests show that the prototype supports the core organisation process. Players were able to view available sessions, open match detail pages, join sessions and leave sessions. These results indicate that the MatchSession and Participation models are functioning correctly and that participation records can be stored and updated. This helps answer RQ1 because structured session management gives organisers a clearer alternative to informal methods such as group chats, verbal confirmation or manual lists.

Attendance testing showed that organiser actions worked as intended. The organiser could mark player attendance, and the attendance value was stored as part of the participation record. This matters for Smart Football because attendance tracking provides more accurate participation data than simply recording who joined a session. It also creates a stronger basis for dashboard statistics and future reliability-based team allocation.

The team allocation and match results tests show that Smart Football extends beyond basic scheduling. The team allocation feature generated Team A and Team B and displayed team-related information for organiser review. The match results page displayed completed match information such as score, MVP and match statistics. These results show that the prototype connects pre-match organisation, team generation and post-match review within one database-driven application.

Taken together, the functional testing results indicate that the prototype is complete enough to demonstrate the intended system concept. The tests show that the main features work together rather than existing as isolated pages. However, the results should still be interpreted within the limits of prototype testing. The tests were carried out in a local development environment using synthetic data, so they show that Smart Football works under controlled conditions rather than proving that it would operate reliably at scale with real users. Even so, the results provide evidence that the prototype meets the core functional requirements of the project.

6.4 Interpretation of Fairness Results

The fairness evaluation suggests that the Smart Football team allocation feature was effective within the scope of the prototype. The generated allocations were compared against a simple join-order baseline, where the first half of joined players were placed into Team A and the second half were placed into Team B. This baseline was useful because it represented a basic non-balancing method similar to an informal manual split. By comparing Smart Football's generated allocation against this baseline, the evaluation could show whether the algorithm improved team balance rather than simply producing teams that appeared acceptable.

The results showed that the generated allocation reduced the difference between Team A and Team B total ratings in seven out of eight evaluated sessions. In one session, Tuesday 5-a-side, the generated difference was the same as the baseline difference. This means that the algorithm generally improved balance but did not improve every case. This is still a useful result because it shows that the allocation method has value without pretending that it is perfect. The strongest improvements were seen in sessions where the baseline split was more uneven, such as Derby Social Football, Riverside Friendly and Late Evening Kickabout. This suggests that the algorithm is most useful when a simple manual split would create noticeably unbalanced teams.

The fairness results also show why using a baseline matters. Without the baseline, the evaluation would only show the final generated team differences. This would make it harder to judge whether Smart Football actually improved the allocation process. The baseline comparison makes the evaluation stronger because it demonstrates that the system can perform better than a simple non-balancing method.

However, the fairness results should be interpreted carefully. The evaluation used synthetic data and stored rating values, so the results depend on the quality and realism of those values. In real football sessions, player performance may be affected by factors that are not fully captured in the prototype, such as fitness on the day, teamwork, player confidence, goalkeeper

availability, defensive balance and preferred playing style. A player's stored rating may also not reflect their actual performance in every match.

The algorithm itself is intentionally lightweight, using a simple greedy balancing approach that organisers can easily review. Additionally, the prototype includes an optional machine learning step using K-Means clustering (via scikit-learn) to group similar players by skill, fitness, experience and position before the greedy distribution. If the required libraries are not installed, the system falls back to the greedy method alone, ensuring the prototype always works. This demonstrates how the allocation could be enhanced without losing the transparency that organisers need.

The fairness evaluation supports the project aim by showing that a lightweight data-driven method can improve team balance compared with a basic manual split. The results provide evidence that Smart Football can support fairer team formation, but the system should be understood as decision support rather than a complete replacement for organiser judgement.

6.5 Interpretation of Usability and Dashboard Findings

The usability review suggests that Smart Football provides a clear and practical interface for the main player and organiser workflows. The prototype uses a consistent dashboard-style layout, visible navigation links, summary cards, tables and action buttons. This helps users understand their current page and identify the actions available to them. In a community football setting, this matters because users need to complete common tasks quickly, including viewing sessions, joining a match, checking profile details, reviewing generated teams and accessing match results.

The player-facing pages support the needs of normal users by giving them access to their own football activity. The player dashboard provides a central place to view joined matches, attendance rate, average rating, upcoming sessions and profile information. This supports the project aim because players are given more visibility over their participation compared with informal organisation methods. Instead of relying on group chat messages or organiser updates, players can use the system to see relevant match and profile information in one place.

The organiser-facing pages support session management by displaying information that would otherwise need to be tracked manually. The organiser dashboard, attendance management page, team allocation page and match results page allow organisers to monitor participation, record attendance, generate teams and review completed matches. This is important because the project is not only about helping players join sessions, but also about reducing the organisational burden on the person managing the football group.

The dashboard visualisations also support the usefulness of the system. By using summary cards, tables and charts, the system presents information in a form that can be understood quickly. This is useful for attendance rates, player numbers, upcoming sessions, team allocation and match results. The dashboards therefore support the third research question, which asks what dashboard information is useful for players and organisers when managing community football participation.

However, the usability findings also have limitations. The review was developer-led rather than based on a large group of real users. This means the evaluation can show that the interface supports the intended workflows, but it cannot fully prove how real players and organisers would

experience the system. Real users may notice issues that were not identified during self-review, such as unclear wording, missing shortcuts, confusing page order or difficulty using the interface on smaller screens.

Another limitation is that some parts of the interface could be improved visually. For example, long dashboard pages may require scrolling, and some permission or error messages may not match the styling of the main interface. These issues do not prevent the prototype from functioning, but they could affect perceived usability and polish. Future development should therefore include real user testing, improved responsive design and more consistent styling for system messages.

The usability and dashboard findings suggest that Smart Football provides a usable foundation for managing community football sessions. The interface supports the main tasks required by players and organisers, while the dashboards make key information easier to access and understand. However, further user testing would be needed to confirm whether the system is genuinely intuitive for real community football users.

6.6 Comparison with Literature

The findings from the Smart Football prototype are consistent with the literature reviewed in Chapter 2. The literature showed that digital technology can support community sport by improving organisation, access and engagement, but that technology must be useful and easy to use if people are expected to adopt it. This is reflected in the prototype results because Smart Football provides structured session management, player profiles, attendance tracking and dashboard information in one system. These features respond to the practical organisational issues identified in community football, where informal methods such as group chats or manual lists can become unreliable as sessions and player numbers increase.

The project also aligns with research on digital sport engagement. Qian and Kim (2025) argue that perceived usefulness, ease of use and adoption barriers influence whether sport participants use digital technology. Harris and Crone (2020) similarly highlight the importance of accessibility, awareness, social influence and practical barriers in technology-enabled physical activity interventions. The usability findings support these ideas because the system was designed to make common tasks visible and simple. Players can view sessions, join matches and check their own information, while organisers can manage attendance, teams and results. This suggests that Smart Football addresses not only the technical problem of storing data, but also the practical need for a system that users can understand.

The fairness evaluation also links to literature on team formation and balancing. The literature review showed that team formation can be treated as a computational problem, where players or participants are grouped using attributes and constraints. Smart Football applies this idea in a lightweight way by using stored player rating data to support team allocation. The results showed that the generated allocation improved team balance in most tested sessions when compared with a simple join-order baseline. This supports the argument that even a simple data-driven approach can improve on informal or arbitrary team selection. However, the findings also reflect the limitations identified in the literature: team formation becomes more complex when additional constraints are considered, such as position, reliability, teamwork, player preference and performance variation.

The fairness findings are also consistent with algorithmic fairness literature. The Centre for Data Ethics and Innovation (2020) argues that fairness is not only about the final outcome, but also about the decision-making process and the data used. Starke et al. (2022) also suggest that perceptions of fairness are shaped by outcome, process, context, features and explanations. Smart Football reflects these ideas by presenting team totals, ratings and allocation information rather than hiding the team generation process completely. The system therefore supports procedural transparency as well as outcome balance. However, the project also confirms that fairness is limited by the quality of the input data. If player ratings are inaccurate or incomplete, the generated allocation may still be flawed.

The use of ratings in Smart Football also relates to research on peer and self-assessment. Power and Tanner (2023) found that peer assessment can be useful, but self-assessment may be less reliable and peer scores can still be inflated. This is relevant because Smart Football uses stored player ratings and profile information to support allocation decisions. The evaluation therefore supports the decision to treat ratings as useful input rather than fully objective truth. In a real deployment, ratings would need careful design, review and moderation to avoid bias or unfair influence on future team allocation.

The dashboard findings connect with dashboard and visualisation literature. Few (2006) describes dashboards as a way of displaying important information so it can be monitored at a glance. Smart Football follows this principle by using player and organiser dashboards to summarise attendance, participation, upcoming sessions, ratings, team allocation and match results. Atterling (2025) also found that interactive visualisation can support football decision-making by improving transparency and reflection. Although Smart Football is much smaller in scope than professional football visualisation tools, the same principle applies, visual summaries help users understand information more quickly and support better decisions.

The comparison with existing sports management platforms also supports the project's practical gap. Platforms such as TeamSnap, Spond and Pitchero show that digital tools are already used for sport organisation, scheduling, communication, membership and payments. However, these systems are generally broader club management platforms rather than focused prototypes for informal football team balancing. Smart Football does not attempt to replace these platforms. Instead, it focuses on a narrower problem by combining scheduling, attendance tracking, dashboard summaries and fair team allocation for small-sided community football.

The implementation findings also connect with the web application framework literature and documentation reviewed earlier. Django was appropriate for this project because the system needed authentication, database models, permissions, templates and form handling. The successful implementation of login, role-based access, database-driven pages and dashboard data supports the decision to use Django as the main framework. The project also reflects the importance of security and data minimisation. Django's authentication and security features supported login, permissions and safer handling of form submissions, while the use of synthetic data and limited stored information followed the principle of reducing unnecessary personal data collection.

The findings support the main themes from the literature review. Digital tools can improve community sport organisation when they are useful, accessible and designed around real user tasks. Team allocation can be improved using data-driven methods, but fairness depends on both outcome balance and transparent process. Dashboards can support decision-making by

presenting important information clearly. Finally, secure web frameworks and data minimisation are important when developing systems that handle user and participation data. Smart Football therefore provides a practical prototype that applies these ideas in the specific context of community football session management.

6.7 Limitations and Threats to Validity

There are several limitations and threats to validity that should be considered when interpreting the results of this project. Although Smart Football achieved its main aim as a functional prototype, the evaluation was carried out in a controlled local environment using synthetic data. This means the results provide useful evidence that the system works as designed, but they do not fully prove how the system would perform in real community football settings.

The first limitation is the use of synthetic data. Synthetic data was useful because it allowed the system to be tested safely without collecting real personal information from players. It also allowed controlled test scenarios to be created for profiles, sessions, attendance, ratings, team assignments and match results. However, synthetic data may not fully represent real user behaviour. In real football sessions, players may cancel at short notice, provide inaccurate self-assessments, forget to update availability, disagree with ratings or have social preferences that affect team selection. These behaviours are difficult to represent accurately in a controlled dataset.

The second limitation is the scale of testing. The prototype was tested with a limited number of users, sessions and team allocation examples. This was suitable for a small-sided football dissertation prototype, but it does not show how the system would behave with a much larger number of users, many sessions running at once, or multiple organisers using the system at the same time. Larger-scale testing would be needed before the system could be considered reliable for real deployment.

A further limitation is that the system was not publicly deployed. Testing was carried out locally using the Django development server and SQLite database. This means the evaluation did not measure real deployment issues such as server performance, network latency, concurrent users, hosting security, backup procedures or long-term database growth. The prototype therefore demonstrates the system concept, but further work would be needed to make it production ready.

The usability evaluation is also limited because it was mainly developer-led. The review showed that the interface supports the intended workflows, but it did not involve a large group of real players or organisers completing tasks independently. As a result, the evaluation may not capture all usability problems. Real users may experience confusion with page layout, wording, navigation, dashboard information or mobile responsiveness in ways that were not identified during self-review.

The fairness evaluation also has limitations. The team allocation algorithm used stored rating values and compared generated teams against a simple join-order baseline. This provided a useful way to measure balance, but it does not capture every factor that affects football fairness. Real match balance can depend on goalkeeper availability, preferred position, teamwork, defensive structure, fitness on the day, player confidence and playing style. The algorithm also depends on the accuracy of profile data and ratings. If users exaggerate their ability or rate others unfairly, the generated allocation may become less reliable.

Subjective ratings are another threat to validity. Ratings can help improve team allocation by giving the system more information about player ability, but they are not fully objective. Players may rate friends more generously; rate others based on personal frustration or misunderstand the rating scale. This means ratings should be treated as decision-support data rather than as a completely accurate measure of player performance.

There is also a technical limitation in the allocation method. The algorithm is intentionally lightweight and transparent, which makes it suitable for a dissertation prototype. The allocation method does not rely entirely on advanced optimisation or machine learning. The main allocation process remains a transparent balancing approach, but the prototype also includes an optional K-Means clustering step that can group similar players before the greedy distribution. If the required libraries are unavailable or the session has too few players for useful clustering, the system falls back to the simpler balancing method.

This limits how many constraints the prototype can handle at once. For example, the system may produce balanced total ratings while still creating teams that are uneven by position. Future versions could improve this by including goalkeeper rules, position constraints, reliability scores and more advanced optimisation methods. Overall, these limitations do not remove the value of the project, but they affect how strongly the results can be generalised. The evaluation shows that Smart Football works as a local prototype and can support fairer team allocation under controlled conditions. However, further testing with real users, real football sessions, deployed infrastructure and larger datasets would be needed to confirm whether the system would be effective in wider community football use.

6.8 Objective Achievement Table

Table 6.1: Achievement of project objectives

Objective	Evidence	Achieved?	Discussion notes
O1 Login/registration	Django authentication, login, logout, registration, profile flow, TC1 and TC2	Yes	The system allowed users to register, log in and access the correct dashboard. Role-based access helped separate player and organiser functionality.
O2 Scheduling	MatchSession model, all sessions page, match details page, join/leave functions, capacity control, TC4, TC5, TC6 and TC7	Yes	Players could view available sessions, open match details, join sessions and leave sessions. This supported structured session management compared with informal manual methods.
O3 Attendance	Participation attendance field, organiser attendance controls, attendance management page, TC9	Yes	Organisers could update attendance records for players. This supported more accurate participation tracking than simply recording who joined a session.
O4 Team allocation	TeamAssignment model, team generation function, fairness comparison	Mostly	The system generated teams and improved team balance in most tested sessions

	table, TC10		compared with a join-order baseline. However, the result depends on rating quality and the algorithm does not account for every football specific factor.
O5 Dashboards/statistics	Player dashboard, organiser dashboard, Chart.js visualisations, match results page, TC3 and TC11	Yes	Dashboards displayed useful information such as participation, attendance, ratings, upcoming sessions and match results. This helped players and organisers access key information quickly.
O6 Evaluation	Functional test table, permission testing, fairness evaluation, usability review, performance checks, TC1 to TC12	Yes	The prototype was evaluated using structured test cases and synthetic data. The evaluation provided evidence for functionality, fairness, usability and performance, although it was limited by local testing and no real user deployment.

7 Conclusions and Recommendations

7.1 Conclusion

This project set out to design and evaluate Smart Football, a data-driven web platform for supporting fair and efficient community football organisation. The project addressed the problem that informal football sessions often depend on manual methods such as group chats, spreadsheets, verbal confirmation or organiser memory. These methods can become unreliable when organisers need to manage player attendance, session capacity, team balance and match information across multiple sessions.

The final prototype demonstrates how a structured web-based system can support both players and organisers. Players can register, log in, manage their football profile, view available sessions, join or leave matches, view team allocations and access match information. Organisers can create sessions, manage attendance, generate teams, view dashboard information and review match results. These features show that Smart Football provides a more organised approach to community football management than informal manual methods.

A key contribution of the project is the fairness-based team allocation feature. The system uses stored player information and rating data to generate teams and compares the generated allocations against a simple join-order baseline. The evaluation showed that the generated allocation improved team balance in most tested sessions, suggesting that a lightweight data-driven method can support fairer team formation for small-sided football. However, the project also recognised that the algorithm is not a complete replacement for organiser judgement because football fairness depends on factors that are not fully captured by the prototype.

Smart Football achieved its main aim within the scope of an academic prototype. Functional testing confirmed that the core workflows operated successfully. Permission testing showed that organiser-only features were protected from normal player accounts. The fairness evaluation provided evidence that the allocation method could improve balance compared with a basic baseline. The usability and performance reviews showed that the system was practical for local small-sided football testing. Further development would be required before real-world deployment, but the project successfully demonstrates the potential of a lightweight web application for community football session management.

7.2 Summary of Contributions

The project made several contributions. First, it produced a working Django-based prototype for community football session management. The system combines authentication, player profiles, match scheduling, attendance tracking, team allocation, dashboards and match results in one application.

Second, the project implemented a lightweight team allocation method that uses stored player data to support fairer team formation. The algorithm is transparent and understandable, making it appropriate for a small community football prototype. It provides decision support for organisers rather than attempting to replace human judgement completely.

Third, the project created player and organiser dashboard pages that summarise useful football information. These dashboards help users view relevant details such as joined matches,

attendance, ratings, upcoming sessions, session activity and match results without needing to check several separate pages.

Fourth, the project included a structured evaluation using synthetic data, test cases, screenshots, permission testing, fairness comparison, usability review and performance checks. This provided evidence that the system worked as intended in a controlled local environment.

Finally, the project identified realistic limitations and future improvements. These include the need for real user testing, deployment testing, stronger optimisation, improved rating reliability, better mobile responsiveness and additional organiser tools.

7.3 Recommendations and Future Work

Future work should first focus on real user testing. The current evaluation used synthetic data and developer-led review, which was suitable for a dissertation prototype. However, testing with real players and organisers would provide stronger evidence about usability, usefulness and fairness perception. Real users may identify issues with navigation, wording, mobile layout or team allocation that are not visible during self-review.

A second improvement would be public deployment. The prototype was tested locally using the Django development server and SQLite database. Future development should deploy the system to a secure hosting environment and test it with multiple users. This would allow performance, reliability, security, backups and database growth to be evaluated more realistically.

The team allocation method could also be improved. The current algorithm is intentionally lightweight and transparent, but future versions could include stronger position balancing, goalkeeper rules, attendance reliability, recent performance, player preferences and manual organiser adjustments. More advanced optimisation methods could also be tested, although these should remain understandable to users.

The rating system could be developed further. Player ratings are useful for team allocation, but they may be subjective or biased. Future work could improve rating reliability by using multiple ratings over time, limiting extreme ratings, adding clearer rating guidance and combining peer ratings with attendance and performance statistics.

The interface could also be improved for mobile use. Many community football users would likely access the system from a phone, so future versions should include stronger responsive design, clearer touch-friendly buttons and reduced scrolling on dashboard pages.

Lastly, additional practical features could be added, such as email reminders, session notifications, payment tracking, waiting lists, recurring sessions and exportable attendance reports. These features were outside the scope of the current prototype, but they would make the system more useful for real community football organisers.

7.4 Final Reflection

This project demonstrated the importance of connecting software development decisions to a clear practical problem. Building Smart Football required more than creating separate pages; the main challenge was making authentication, profiles, sessions, attendance, ratings, team allocation, dashboards and results work together through a shared database structure.

The project also showed that fairness in software is not only a technical issue. Although the team allocation algorithm could reduce rating differences between teams, fairness also depends on the quality of player data, user trust, transparency and organiser judgement. This was an important lesson because a system can produce a balanced numerical result while still needing human review.

Developing the prototype also strengthened understanding of Django, database relationships, role-based access, template design, testing and evaluation. The project required practical decisions about scope, especially because adding too many features could reduce the quality of the final system. Focusing on the core objectives helped keep the prototype realistic.

The project provided useful experience in designing, implementing and evaluating a software artefact for a real-world problem. Smart Football is not a finished commercial product, but it successfully demonstrates how a web-based system can support more organised and transparent community football session management.

8 References

Alman, J. and McKay, D. (2017) 'Theoretical foundations of team matchmaking', Proceedings of the 16th International Conference on Autonomous Agents and Multiagent Systems (AAMAS 2017), pp. 1073–1081.

Atterling, F. (2025) From Data to Strategy: Designing and Evaluating an Interactive Visualization Tool for Football Club Decision-Making. KTH Royal Institute of Technology.

Aysolmaz, B., Müller, R. and Meacham, D. (2023) 'The public perceptions of algorithmic decision-making systems: Results from a large-scale survey', *Telematics and Informatics*, 79, 101954. doi: 10.1016/j.tele.2023.101954.

Barnabò, G., Fazzino, A., Leonardi, S. and Schwiegelshohn, C. (2019) 'Algorithms for fair team formation in online labour marketplaces', *Companion Proceedings of The 2019 World Wide Web Conference*, pp. 484–490. doi: 10.1145/3308560.3317587.

Beal, R., Norman, T.J. and Ramchurn, S.D. (2019) 'Artificial intelligence for team sports: a survey', *The Knowledge Engineering Review*, 34, e28. doi: 10.1017/S0269888919000225.

Beck, K. et al. (2001) Principles behind the Agile Manifesto. Available at: <https://agilemanifesto.org/principles.html> (Accessed: 3 February 2026).

Centre for Data Ethics and Innovation (2020) Review into bias in algorithmic decision-making. London: Centre for Data Ethics and Innovation. Available at: <https://www.gov.uk/government/publications/cdei-publishes-review-into-bias-in-algorithmic-decision-making> (Accessed: 4 February 2026).

Chart.js (2025) Getting Started. Available at: <https://www.chartjs.org/docs/latest/getting-started/> (Accessed: 5 February 2026).

Django (2026a) Using the Django authentication system. Available at: <https://docs.djangoproject.com/en/6.0/topics/auth/default/> (Accessed: 5 February 2026).

Django (2026b) User authentication in Django. Available at: <https://docs.djangoproject.com/en/6.0/topics/auth/> (Accessed: 5 February 2026).

Django (2026c) Security in Django. Available at: <https://docs.djangoproject.com/en/6.0/topics/security/> (Accessed: 6 February 2026).

Few, S. (2006) Information Dashboard Design: The Effective Visual Communication of Data. Sebastopol: O'Reilly Media.

Harris, M.A. and Crone, D. (2020) 'Motivations and barriers to engagement with a technology-enabled community wide physical activity intervention', PLOS ONE, 15(6), e0232317. doi: 10.1371/journal.pone.0232317.

Idris, N., Mohd Foozy, C.F. and Shamala, P. (2020) 'A Generic Review of Web Technology: Django and Flask', International Journal of Advanced Science Computing and Engineering, 2(1), pp. 34–40.

Information Commissioner's Office (n.d.) Principle (c): Data minimisation. Available at: <https://ico.org.uk/for-organisations/uk-gdpr-guidance-and-resources/data-protection-principles/a-guide-to-the-data-protection-principles/data-minimisation/> (Accessed: 6 February 2026).

Kumar, M. and Nandal, R. (2024) 'Role of Python in Rapid Web Application Development Using Django', SSRN Electronic Journal. doi: 10.2139/ssrn.4751833.

Mozilla (2025) JavaScript. Available at: <https://developer.mozilla.org/en-US/docs/Web/JavaScript> (Accessed: 7 February 2026).

Pitchero (n.d.) Sports Club Management Software. Available at: <https://www.pitchero.com/> (Accessed: 7 February 2026).

Power, J.R. and Tanner, D. (2023) 'Peer assessment, self-assessment, and resultant feedback: an examination of feasibility and reliability', European Journal of Engineering Education, 48(4), pp. 615–628. doi: 10.1080/03043797.2023.2185769.

Python (2026a) sqlite3 — DB-API 2.0 interface for SQLite databases. Available at: <https://docs.python.org/3/library/sqlite3.html> (Accessed: 8 February 2026).

Python (2026b) venv — Creation of virtual environments. Available at: <https://docs.python.org/3/library/venv.html> (Accessed: 8 February 2026).

Qian, R. and Kim, K. (2025) 'The Intention of Sports Participants to Utilize Digital Technology for Engagement: The Moderating Role of Self-Efficacy', *Behavioral Sciences*, 15(3), 367. doi: 10.3390/bs15030367.

Recalde, D., Severín, D., Torres, R. and Vaca, P. (2018) 'An exact approach for the balanced k-way partitioning problem with weight constraints and its application to sports team realignment', *Journal of Combinatorial Optimization*, 36(3), pp. 916–936. doi: 10.1007/s10878-018-0254-1.

Sanchez-Anguix, V., Alberola, J.M., Del Val, E., Palomares, A. and Teruel, M.D. (2023) 'Comparing computational algorithms for team formation in the classroom: a classroom experience', *Applied Intelligence*, 53, pp. 23883–23904. doi: 10.1007/s10489-023-04748-5.

Schwaber, K. and Sutherland, J. (2020) *The Scrum Guide*. Available at: <https://scrumguides.org/scrum-guide.html> (Accessed: 8 February 2026).

Spond (n.d.) Spond App. Available at: <https://www.spond.com/> (Accessed: 9 February 2026).

Sport England (2026) England is getting more active but not everyone is benefiting. Available at: <https://www.sportengland.org/> (Accessed: 9 February 2026).

SQLite (2025) About SQLite. Available at: <https://www.sqlite.org/about.html> (Accessed: 9 February 2026).

Starke, C., Baleis, J., Keller, B. and Marcinkowski, F. (2022) 'Fairness perceptions of algorithmic decision-making: A systematic review of the empirical literature', *Big Data & Society*, 9(2), pp. 1–16. doi: 10.1177/20539517221115189.

TeamSnap (n.d.) Team Management App Features. Available at: <https://www.teamsnap.com/> (Accessed: 10 February 2026).

The Football Association (2024) *A Thriving Grassroots Game: Grassroots Strategy 2024–2028*. London: The Football Association. Available at: <https://www.thefa.com/football-rules-governance/policies/grassroots-football-strategy> (Accessed: 10 February 2026).

Vombatkere, K., Gionis, A. and Terzi, E. (2025) 'Forming coordinated teams that balance task coverage and expert workload', *Data Mining and Knowledge Discovery*, 39(3), article 19. doi: 10.1007/s10618-025-01090-x.

9 Appendices

9.1 Appendix A: User Manual / Demo Guide

Purpose of the User Manual

This appendix provides a short user manual and demonstration guide for the Smart Football prototype. The purpose is to show how the system can be used by two main user types: a player and an organiser. The guide explains how to run the system locally, which demo accounts to use, and which core features should be shown during a prototype demonstration.

Running the System Locally

The prototype was developed using Django and runs locally in a browser.

To start the system:

1. Open the project folder in Visual Studio Code.
2. Open the terminal.
3. Activate the virtual environment:

```
.venv\Scripts\activate
```

4. If demo data is needed (Should already be loaded), run:

```
python manage.py seed_demo_data
```

5. Start the Django development server:

```
python manage.py runserver
```

6. Open the system in a browser:

```
http://127.0.0.1:8000/
```

The prototype should then display the Smart Football login or dashboard page.

Demo Accounts

The following two accounts are used for the demonstration.

User type	Username	Password	Purpose
Organiser	organiser_james	password123	Used to create/manage sessions, mark attendance, generate teams, and view organiser dashboard information.
Player	amelia_brown	password123	Used to show the player dashboard, profile details, session joining, attendance history, and team information.

These accounts are for local demonstration only and should not be used in a real deployed system.

Player User Guide

This section describes how a player uses the Smart Football system.

1. Step 1: Log in as a player

1. Open the Smart Football website.
2. Go to the login page.
3. Enter the following details:

Username: amelia_brown

Password: password123

4. Click **Login**.

Expected result: the player is taken to the player dashboard.

2. Step 2: View the player dashboard

After logging in, the player dashboard displays a summary of the player's activity. This may include:

- matches joined
- attendance rate
- average rating
- upcoming sessions
- profile summary
- team allocation information where available

This page helps the player quickly understand their current football activity.

3. Step 3: View available sessions

1. Click **All Sessions** in the navigation/sidebar.
2. Review the list of available football sessions.
3. Check session details such as:
 - session title
 - date and time
 - location
 - capacity
 - joined players
 - session status

Expected result: the player can see which football sessions are available.

4. Step 4: Join a session

1. Choose a session that is not full.
2. Click **Join** or open the session details page and click **Join Session**.

Expected result: the system adds the player to the session and updates the joined count.

5. Step 5: View match details

1. Click on a session from the 'all sessions page'.
2. View the match details page.

The match details page may show:

- date and time
- location
- capacity
- participants
- attendance summary
- generated teams
- rating or result options, if available

Expected result: the player can view relevant information about the selected match.

Organiser User Guide

This section describes how an organiser uses the Smart Football system.

6. Step 1: Log in as an organiser

1. Open the Smart Football website.
2. Go to the login page.
3. Enter the following details:

Username: organiser_james

Password: password123

4. Click **Login**.

Expected result: the organiser is taken to the organiser/dashboard area.

7. Step 2: View organiser dashboard

The organiser dashboard provides a summary of sessions created or managed by the organiser. It can show information such as:

- created sessions
- number of joined players
- attendance count

- upcoming or past sessions
- dashboard statistics

Expected result: the organiser can monitor session activity from one place.

8. Step 3: Create a new session

1. Click **Create Match** or **Create Session**.
2. Enter the session details:
 - title
 - date and time
 - location
 - capacity
3. Click **Save** or **Create**.

Expected result: the new session is added to the system and appears in the session list.

9. Step 4: Manage attendance

1. Open a session created by the organiser.
2. Go to the attendance management page or attendance controls.
3. Mark players as:
 - attended
 - not attended / not marked

Expected result: attendance records are updated and reflected in the session statistics.

10. Step 5: Generate teams

11. Open a match session with enough joined players.
12. Click **Generate Teams** or open the Team Allocation page.
13. Review Team A and Team B.

The team allocation page may show:

- players assigned to each team
- player scores
- team totals
- balance/fairness information

Expected result: the system creates two teams using stored player profile information such as skill level, fitness level, and experience.

14. Step 6: Review match results

For completed matches, the organiser can view match result information where available. This may include:

- final score
- team statistics
- player match statistics
- MVP
- fairness score

Expected result: the organiser can review the match outcome and supporting statistics.

9.2 Appendix B: Screenshots and Wireframes

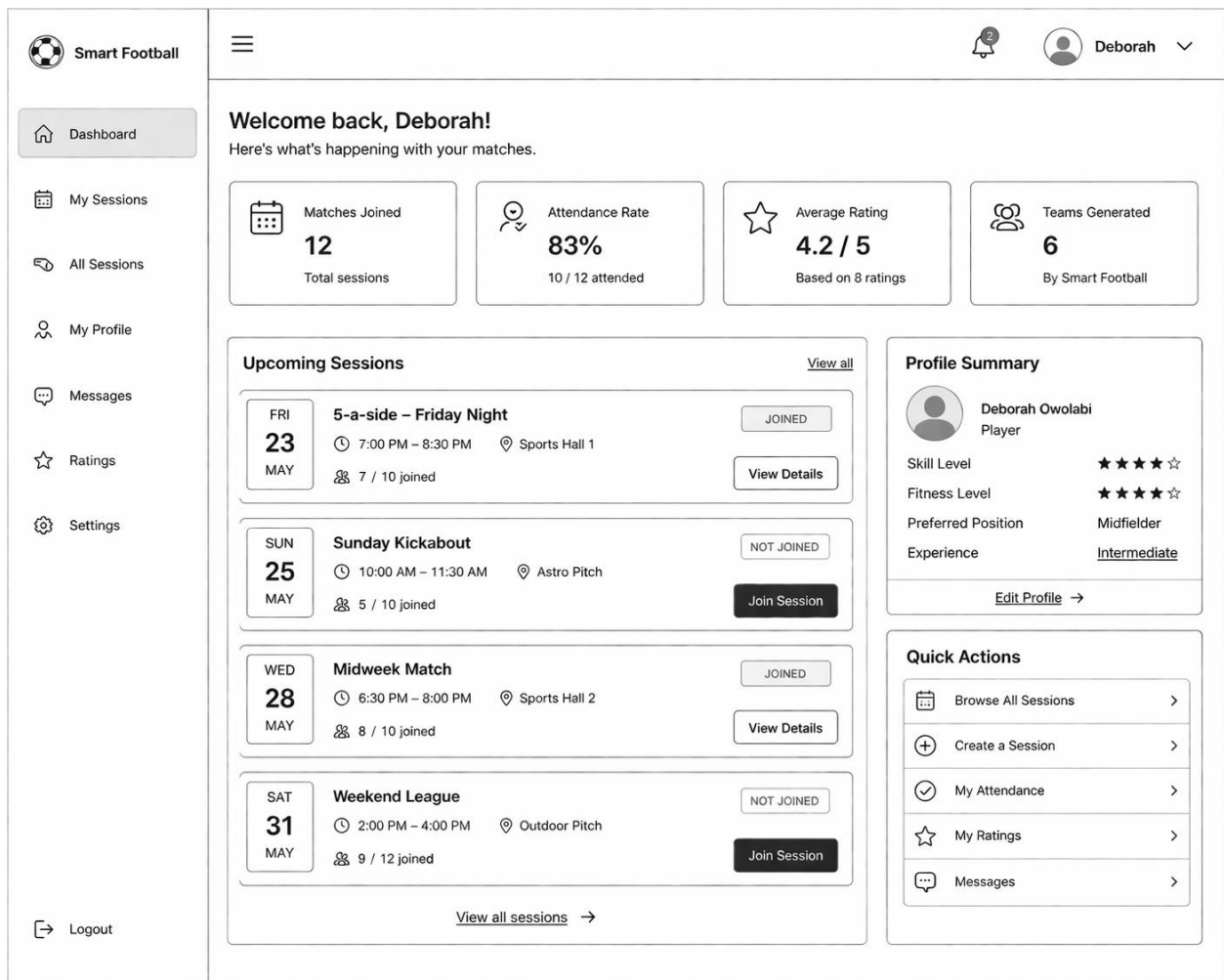


Figure 9.2.1: Player dashboard Wireframe

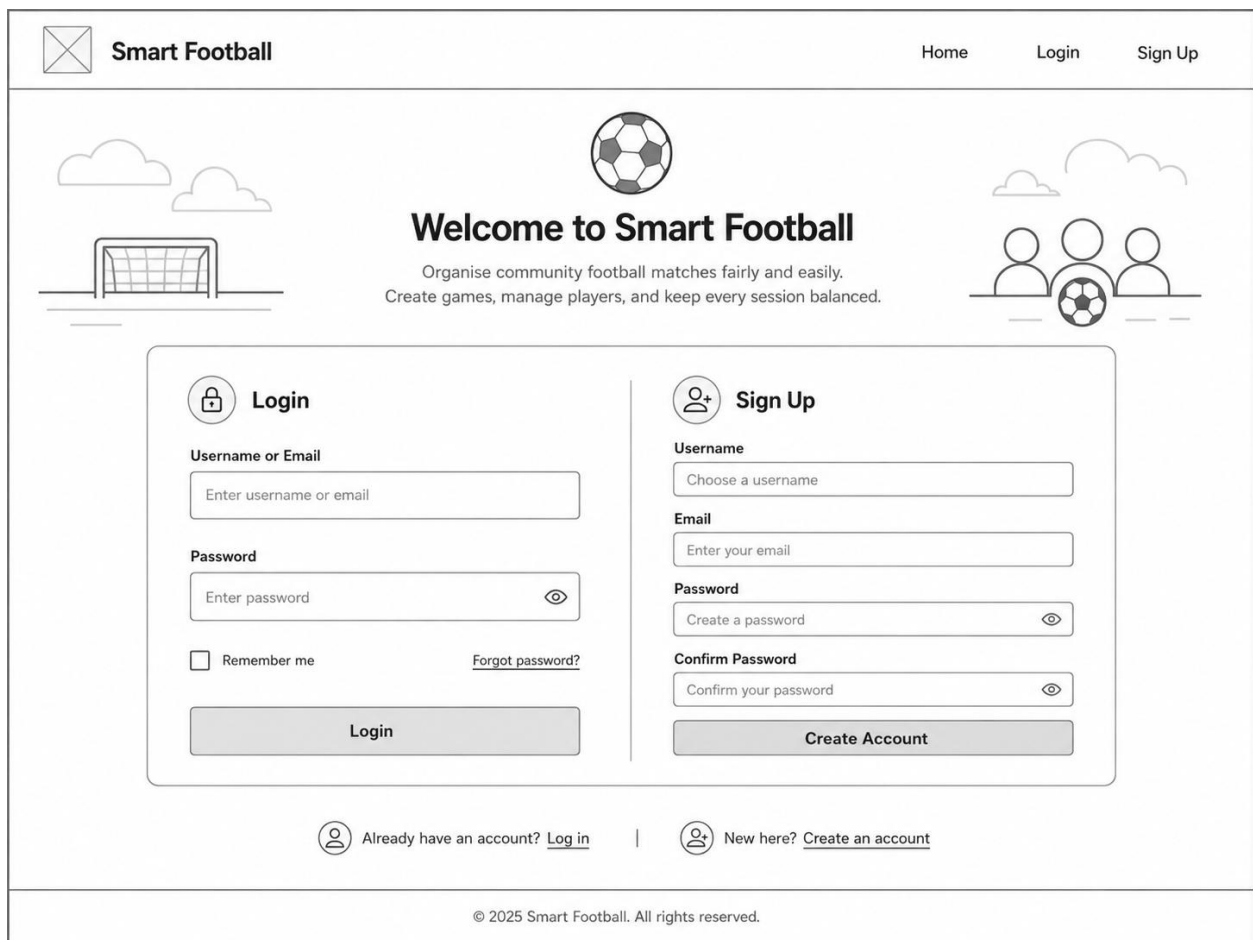


Figure 9.2.2: Login/Signup wireframe

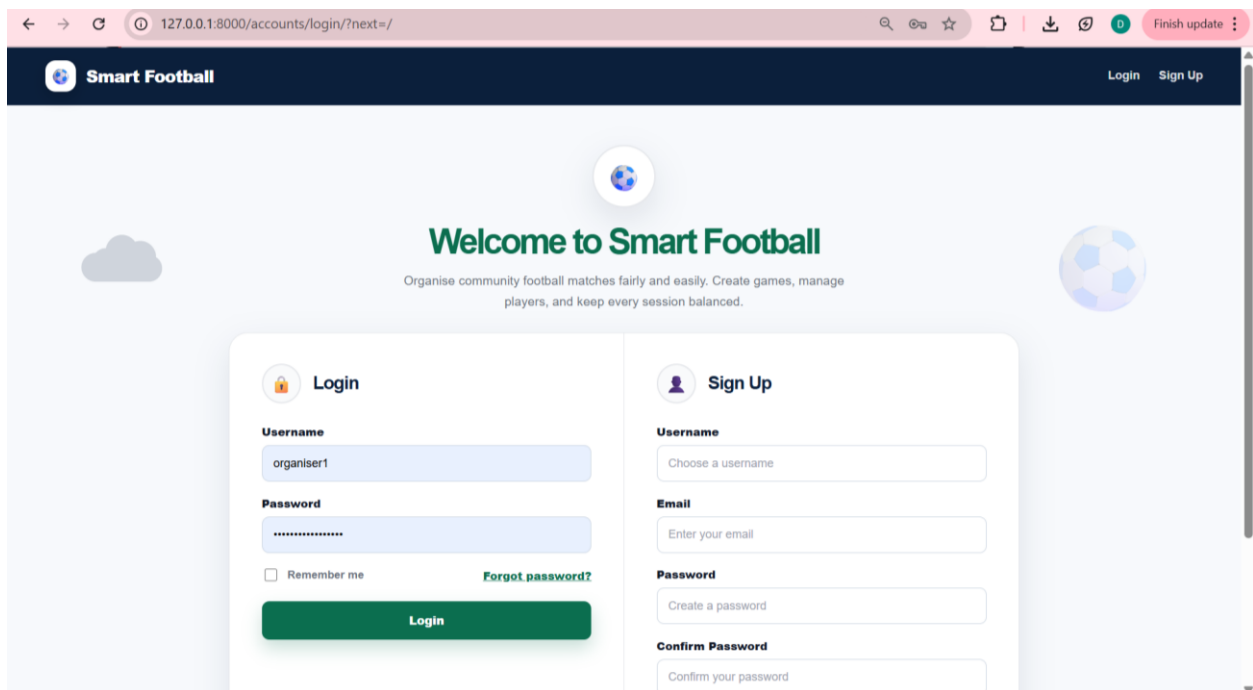


Figure 9.2.3: Login/Signup page

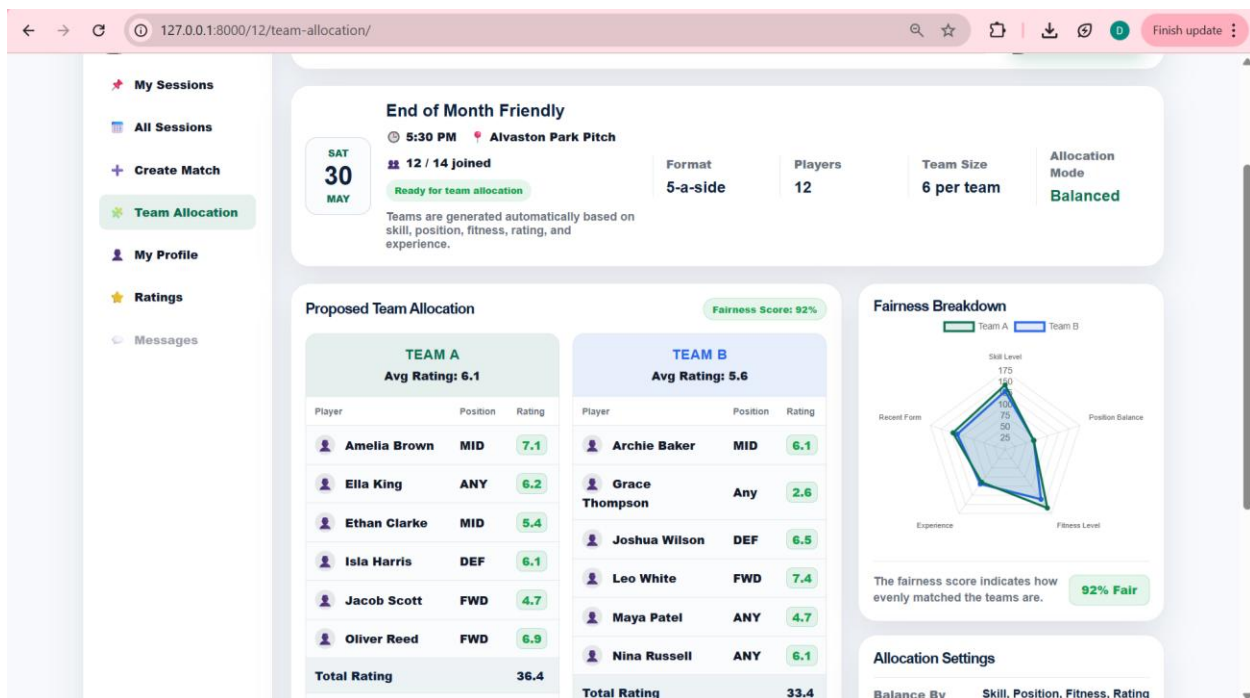


Figure 9.2.4: Team Allocation page

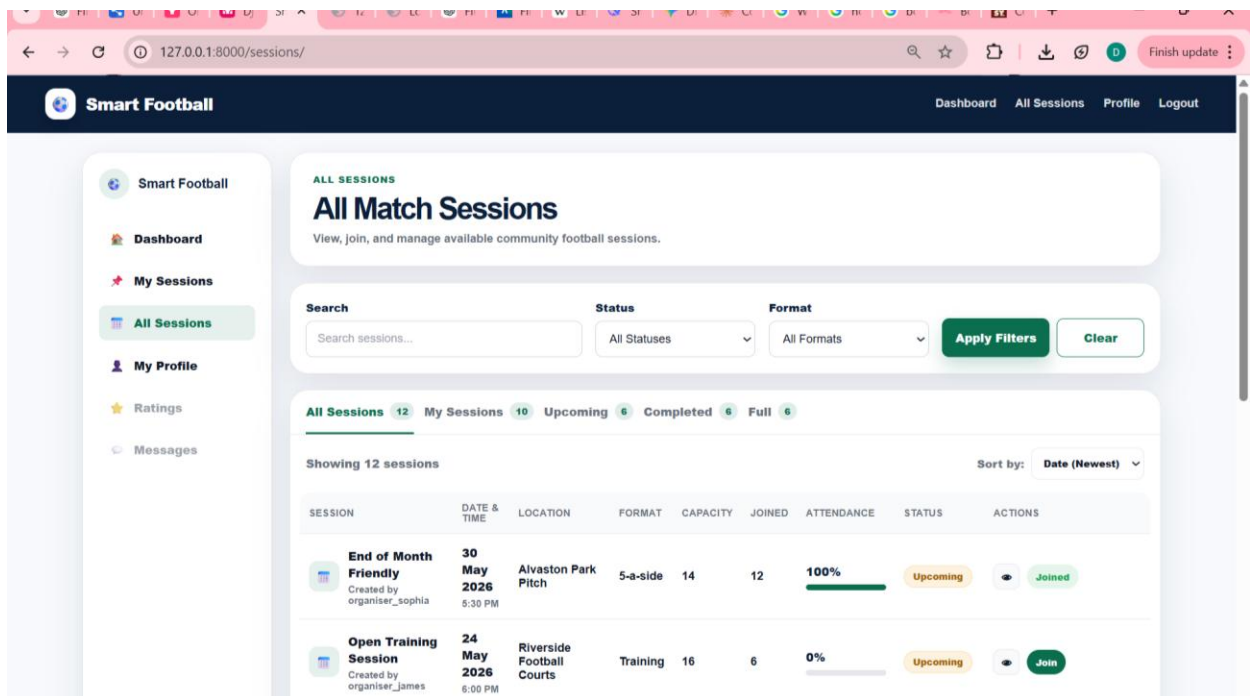


Figure 9.2.5: All sessions page showing available matches, filters, session status

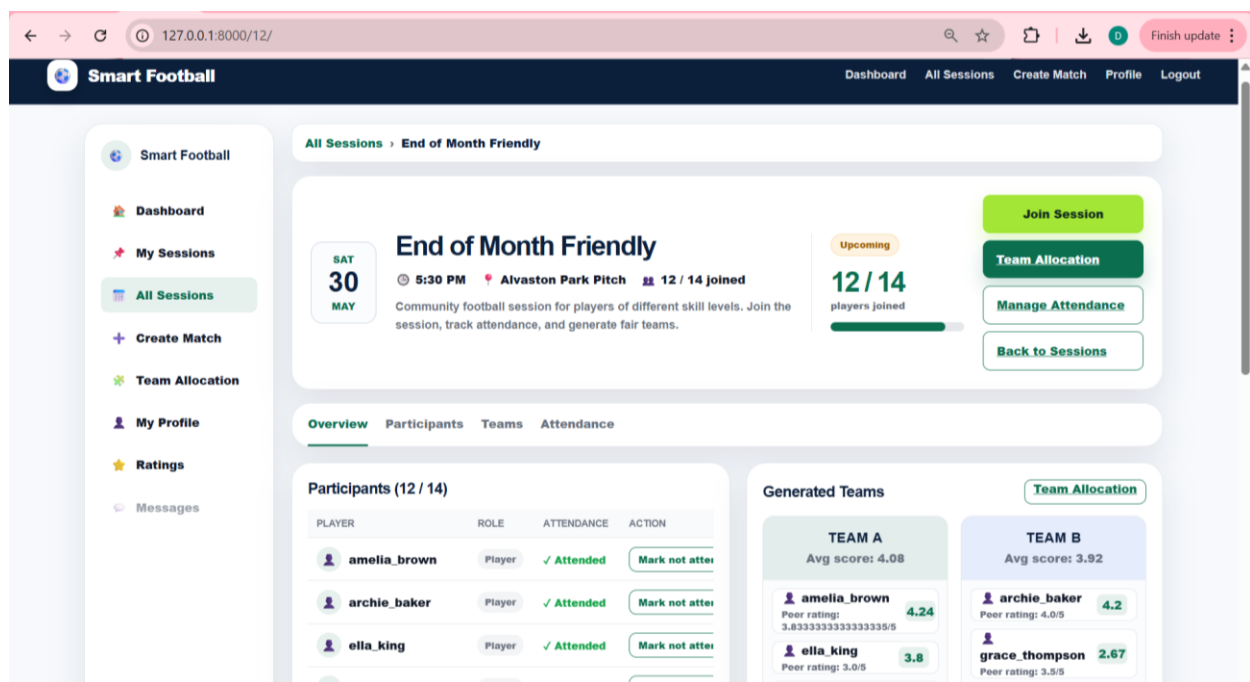


Figure 9.2.6: Match details page showing session information, capacity, participants, attendance summary

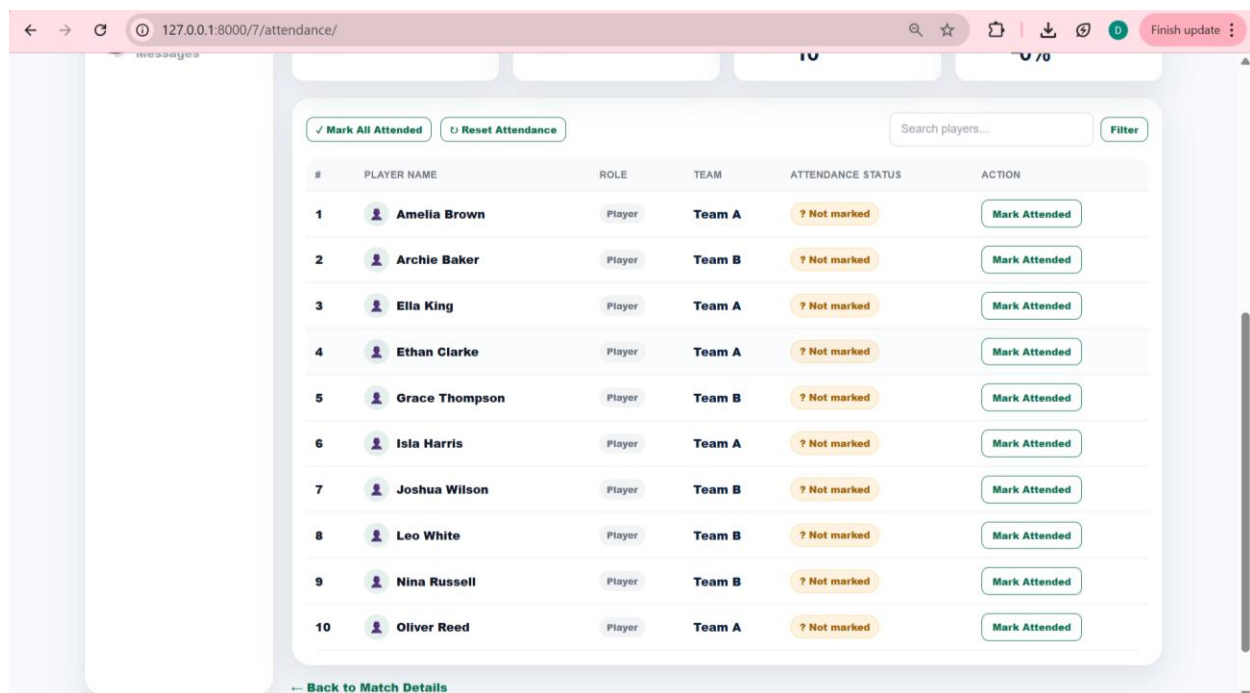


Figure 9.2.7: Attendance management page showing player attendance statuses and organiser attendance controls

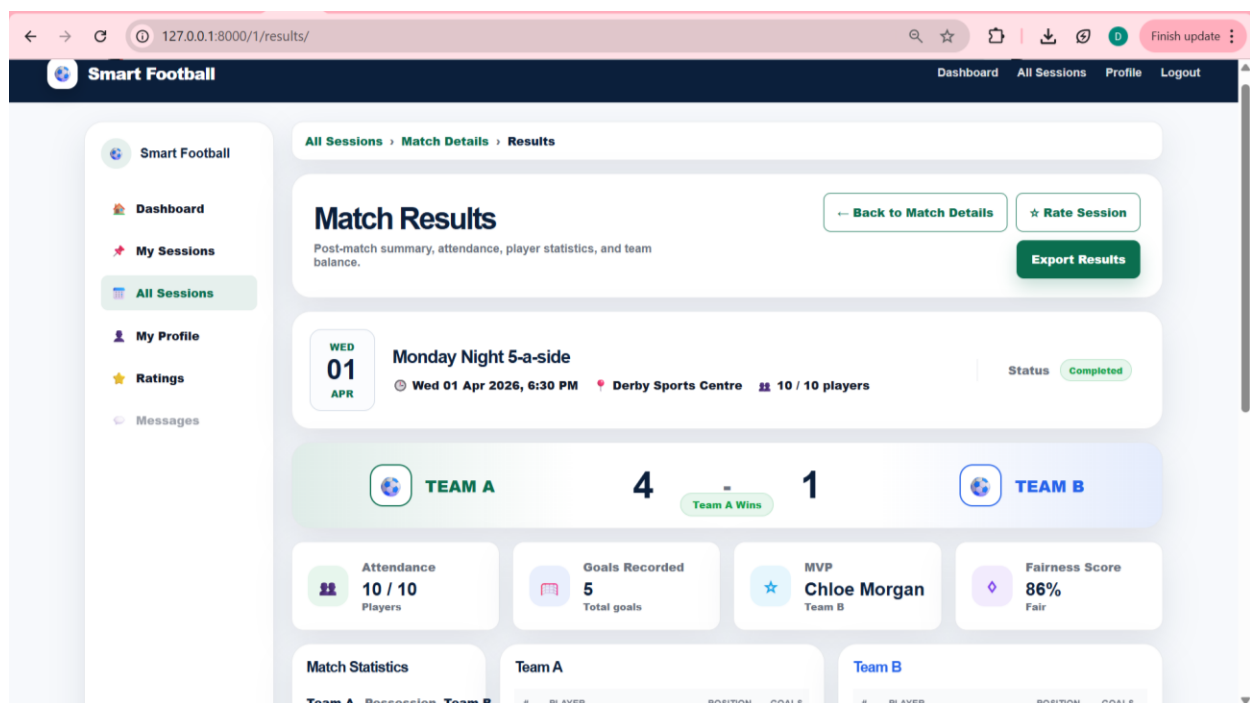


Figure 9.2.8: Match results page showing final score, attendance count, MVP, fairness score and match statistics

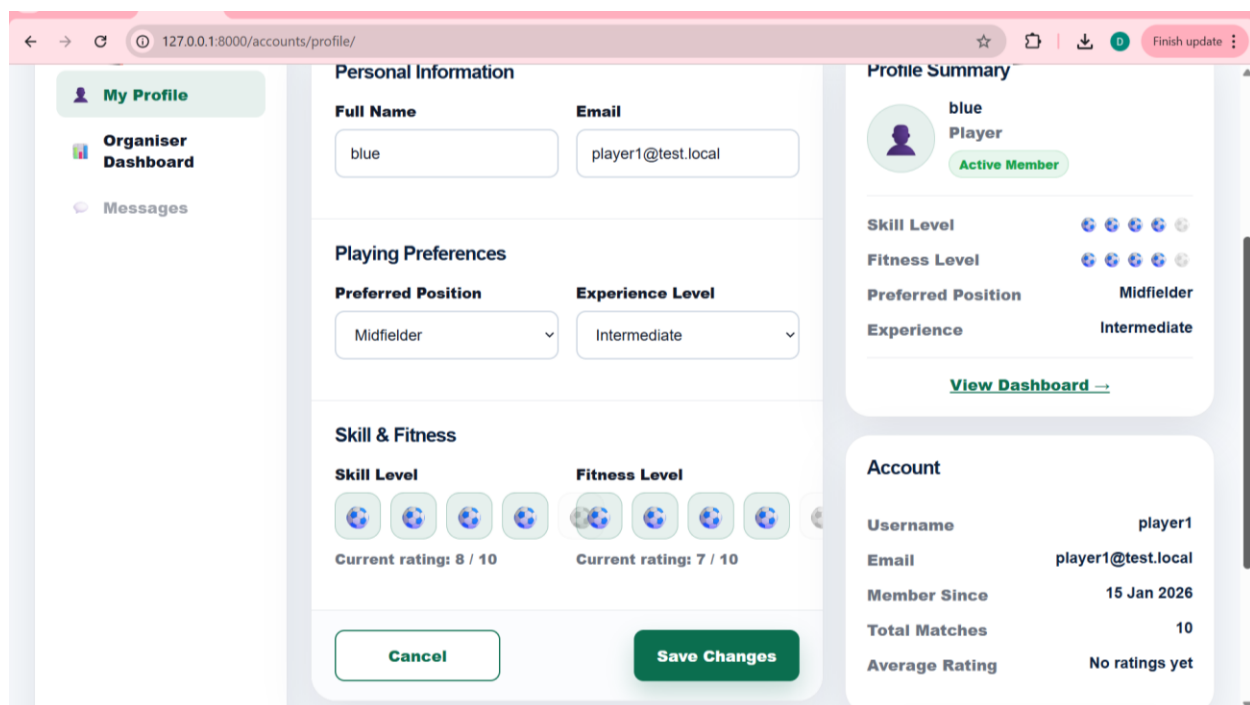


Figure 9.2.9: Edit profile page showing football profile fields such as position, skill level, fitness level and experience.